



本科毕业设计（论文）

基于深度学习的番茄叶片病虫害检测系统的设计与实现

学院（部、中心）： 软件学院

专 业： 软件工程

班 级： 软件工程 2103

学生姓名： 谢曼思

学 号： 2214424316

指导教师： 李晨

2025 年 06 月

摘 要

番茄是全球重要的经济作物之一，广泛种植于我国多个地区。但在其生长过程中易受多种害虫侵扰，影响产量与品质。当前常用的人工识别方法效率低、依赖经验，准确率不高。随着深度学习技术的发展，基于图像识别的自动检测手段为虫害识别提供了新思路。针对番茄病害识别模型对存储和计算资源依赖性强的问题，本课题提出了一种轻量化的 Light-YOLOv8 模型，并基于该模型设计并实现了一个番茄叶片病虫害检测系统。

针对 YOLOv8n 网络在番茄病虫害检测应用中的计算复杂性问题，设计了一种融合 Ghost 组件与 L1 范数剪枝技术的网络轻量化策略。通过 Ghost 模块及 C3Ghost 组件对原始主干网络进行重构，利用 Ghost 模块的低成本线性运算显著降低了计算负担。同时运用 L1 范数标准识别重要卷积核，剔除对检测性能影响较小的特征通道。经过多轮微调优化的 Light-YOLOv8 网络，在平均精度仅降低 1.1%的前提下，达到了帧率增长 35.5%、运算量削减 54.7%的优异表现，为计算资源有限环境中的实时病害识别提供了有效技术方案。结合农业应用的实际业务需求，本文对番茄叶片病虫害检测平台进行了需求调研与架构设计。平台功能架构包含三大主要模块：病害识别检测模块、历史数据管理模块以及用户权限管理模块，通过类关系图与时序图详细描述了各功能模块的设计细节，并运用 Flask 技术框架实现了算法模型的有效部署。在技术选型方面，采用 SpringBoot 后端与 Vue 前端的混合开发模式，保障了系统运行的可靠性与界面交互体验。最后，对整个平台的各项功能模块进行了完整的测试验证。

本系统有效完成了病害检测、算法集成、数据筛选及批量导出等核心功能。所整合的 Light-YOLOv8 算法在识别准确率与处理效率方面表现出色，各功能模块皆通过完整的测试验证。后续工作将重点关注平台的持续改进，包括引入多用户并发处理、扩展数据库容量、完善数据归档流程等方面，从而提高平台的应用价值和拓展潜力。

关 键 词：番茄叶片病虫害；YOLOv8；轻量化模型；检测系统

ABSTRACT

Tomato is one of the most important economic crops worldwide and is widely cultivated in many regions of China. However, during its growth, it becomes susceptible to various pest infestations. These infestations affect both yield and quality. The commonly used manual identification methods are inefficient, experience-dependent, and not highly accurate. With the development of deep learning technologies, image-based automatic detection provides a new approach for pest identification. Tomato disease recognition models typically rely heavily on storage and computational resources. To address this issue, in this thesis, a lightweight model named Light-YOLOv8 is proposed. Based on this model, a tomato leaf pest and disease detection system is designed and implemented.

In this thesis, the computational burden of YOLOv8n in tomato leaf disease recognition is addressed by designing an optimization strategy. This strategy combines lightweight network structure and channel pruning. The improvement scheme adopts lightweight Ghost components to replace the original backbone network parts. It utilizes simplified computational characteristics to significantly reduce resource consumption. Meanwhile, weight importance assessment methods are used to screen core parameters. These methods remove channels with minimal contribution to recognition effectiveness. After repeated training adjustments, the streamlined Light-YOLOv8 model successfully increases processing speed by 35.5%. It also reduces computational load by 54.7%, while recognition accuracy decreases by only 1.1%. This provides a practical solution for hardware-constrained environments. Combining actual agricultural application needs, the disease detection platform's requirement analysis and architecture design are completed. The system is divided into three major functional modules: recognition query, data management, and user permissions. UML diagrams clarify the system's operational logic. A lightweight service framework is adopted to deploy the recognition model. Mainstream front-end and back-end technology stacks are used to build the user interface. This ensures system stability and interactive experience. All functions have been verified through comprehensive testing.

The platform implements core functions such as image recognition, result analysis, and data management. The integrated lightweight algorithm performs excellently in recognition effect and response time. Future work will further improve system scalability. This includes optimizing concurrent processing capabilities and strengthening data management mechanisms to enhance overall practical value.

KEY WORDS: Tomato leaf pests and diseases; YOLOv8; Lightweight model; Detection system

目 录

摘 要.....	I
ABSTRACT.....	II
1 绪论.....	1
1.1 研究背景和意义.....	1
1.2 国内外研究现状.....	1
1.2.1 基于机器学习的国内外研究.....	1
1.2.2 基于深度学习的国内外研究.....	2
1.3 论文主要内容.....	3
1.4 论文组织结构.....	4
2 相关理论与技术.....	5
2.1 基于深度学习的目标检测算法.....	5
2.2 系统开发 Web 技术.....	6
2.2.1 后端开发技术.....	6
2.2.2 前端开发技术.....	7
2.3 本章小结.....	8
3 基于改进 YOLOv8n 的番茄叶片病虫害检测模型.....	9
3.1 应用场景分析与问题描述.....	9
3.2 基于 Ghost 模块的卷积轻量化改进.....	9
3.3 基于剪枝的模型轻量化改进.....	13
3.4 实验设计和结果分析.....	16
3.4.1 实验数据集及实验环境.....	16
3.4.2 实验结果评价指标.....	17
3.4.3 对比实验分析.....	18
3.4.4 消融实验分析.....	18
3.5 本章小结.....	19
4 番茄叶片病虫害识别系统的需求与设计.....	20
4.1 系统需求分析.....	20
4.1.1 系统功能性需求分析.....	20
4.1.2 系统非功能性需求分析.....	21
4.2 系统概要设计.....	22
4.2.1 系统整体体系结构.....	22
4.2.2 系统功能模块设计.....	23
4.3 系统详细设计.....	24

4.3.1 识别搜索模块设计	24
4.3.2 识别记录管理模块设计	25
4.3.3 用户信息管理模块设计	27
4.4 数据库设计	29
4.5 本章小结	32
5 番茄叶片病虫害识别系统的实现与测试	33
5.1 系统开发环境	33
5.2 系统功能模块实现	33
5.2.1 识别搜索模块实现	33
5.2.2 识别记录管理模块实现	37
5.2.3 用户信息管理模块实现	43
5.3 系统测试	45
5.3.1 测试环境	45
5.3.2 功能测试	46
5.3.3 非功能性测试	47
5.4 本章小结	49
6 结论与展望	50
6.1 结论	50
6.2 展望	51
致 谢	52
参考文献	53

1 绪论

1.1 研究背景和意义

在我国农业发展过程中，番茄作为种植面积广、经济价值高的重要蔬菜作物，扮演着促进农民增收、保障蔬菜供应的重要角色。番茄生长周期长、管理要求高，且容易受到多种病虫害的侵袭，轻则影响产量与品质，重则导致大面积减产甚至绝收。为了有效识别和控制病虫害，保障番茄产业的可持续发展，番茄病虫害检测成为了种植管理中不可或缺的重要环节。目前传统的检测方法主要有两种：一是依靠农户或农技人员的经验进行目测识别，但此方式受限于个人经验差异大、判断标准不一，容易出现识别误差；二是通过常规施药手段进行预防，虽然农药能防治番茄病虫害，但滥用会破坏生态环境，违背绿色农业理念，不利于其可持续发展。

番茄叶片作为病虫害最早发生和最易观测的部位，其表面纹理、色泽及形态特征的变化往往能直接反映病害的种类和程度。与健康叶片相比，受病虫害侵扰的区域在纹理图像中通常表现为局部像素值突变或不规则的边缘特征，为基于图像识别技术进行检测提供了重要依据。由于自动检测系统具有低人力成本、检测速度快、可大规模推广等优点，近年来已成为农业信息化领域的重要研究方向。

传统的基于图像处理的检测方法，如 Sobel 算子、Canny 边缘检测器等，虽在理想环境下能够实现较高精度，但在实际应用中，番茄叶片常因拍摄角度、光照变化、背景干扰等因素导致边界模糊、噪声增加，使得常规方法难以稳定工作。此外，基于固定阈值的分割方法虽然简单直观，但对光照、色差的变化极为敏感，容易出现漏检或误检的情况。面对复杂自然环境下的实际检测需求，仅依靠传统图像处理方法已难以满足高准确率与强鲁棒性的要求。近年来，卷积神经网络(CNN)的发展为病虫害识别注入了新活力。CNN 能够通过多层次特征提取，自适应捕捉病虫害区域的复杂纹理和形态变化，显著提升了病虫害检测的准确性与泛化能力。

本文结合深度学习技术，围绕番茄叶片病虫害识别展开研究，构建基于 YOLOv8 模型的自动检测系统，可帮助农户高效识别病害并精准防治，保障产量同时为智慧农业发展提供技术支持。

1.2 国内外研究现状

农业病虫害识别技术发展大致经历了三个阶段：人工观察、机器学习识别，以及深度学习识别。

1.2.1 基于机器学习的国内外研究

在数字化农业发展的背景下，作物病害检测技术逐步从传统方法向智能化方向转变。早期的病害识别系统主要依托于经典的机器学习框架，其核心技术流程可概括为

多个连续的处理环节：初始阶段需要对原始图像进行标准化预处理，确保数据质量的一致性；紧接着运用计算机视觉中的图像分割方法，实现病害区域的准确定位与分离；在此基础上，通过多种特征提取算法获得图像的描述性特征，并结合专家知识进行人工特征选择与降维处理；最终构建基于传统机器学习的分类模型，通过监督学习的方式训练分类器，使其具备对不同病害类型进行准确判别的能力。

传统机器学习技术在农作物病害检测领域经历了重要的技术演进过程，众多研究者在此方向上贡献了创新性成果。早期的探索工作中，Pujari 等人^[1]于 2013 年聚焦于谷物病害的视觉特征分析，设计了融合 K-means 无监督聚类与支持向量机分类器的混合算法框架，在谷物真菌性病害的自动识别任务中取得了显著效果。随后，张云龙等研究者^[2]在 2017 年的工作中引入了改进型 mean-shift 分割算法，以差分直方图统计特征和病斑颜色描述符为主要依据，开发了针对苹果叶部病害的高效识别框架，该系统的分类性能超过了 96% 的准确率阈值。在特征优化方面，蒲一鸣^[3]于 2019 年提出了基于主成分分析的特征降维策略，通过对比反向传播神经网络与支持向量机的分类效果，确定了 BP 网络在水稻病害识别任务中的优越性能。针对复杂环境下的病害检测挑战，Abdulridha 等科研团队^[4]在 2019 年设计了多层感知机架构，该模型展现出在环境噪声干扰情况下稳定检测鳄梨月桂枯萎病的技术能力。近期研究中，Soarov 等学者^[5]于 2021 年提出了结合 Otsu 动态阈值与直方图均衡化的图像预处理流程，通过精准的病害区域提取配合支持向量机学习算法，实现了 96% 识别精度的苹果病害检测系统。

1.2.2 基于深度学习的国内外研究

在当前的技术发展阶段，卷积神经网络（Convolutional Neural Network, CNN）已经确立了其在特征提取领域的核心地位。相关研究成果表明：ZENG 等学者^[6]将 CNN 技术应用于果蔬分类任务中，基于自构建数据集实现了 95.6% 的分类准确率；毛锐等研究者^[7]针对 ResNet-50 架构进行了结构优化，通过引入卷积核分解技术和 ROI Align 算法改进策略，使得改进后的 Faster-RCNN 模型在小麦病害识别任务中的检测精度达到 98.74%；Shi 等人^[8]提出了一种融合传统卷积操作与 Ghost 瓶颈结构的轻量化模型设计方案，同时采用 GELU 激活函数替换传统的 ReLU 函数，有效提升了模型的识别准确性；Li 等研究团队^[9]对 MobileNetV2 网络架构进行了针对性改进，成功实现了植物叶片的智能识别，其识别准确率达到 91.41%。

虽然当前的卷积神经网络架构通过深度网络结构的扩展实现了模型性能的持续改进，但这种发展趋势同时带来了计算复杂度和存储空间需求的急剧上升问题。具体表现为：模型训练过程的时间成本大幅增加，网络参数规模呈现指数级增长态势，这些技术瓶颈严重制约了深度学习模型在资源受限环境下的实际部署，特别是在移动设备和嵌入式系统等边缘计算场景中的广泛应用。

针对上述技术挑战，研究者们转向探索单阶段目标检测框架，其中 SSD (Single Shot Multibox Detector)^[10]和 YOLO (You Only Look Once) 系列算法^[11]凭借其优异的计算效率和实时处理能力，在复杂自然环境中的目标识别与精确定位任务中获得了广泛关

注。相关应用研究显示：CHEN 等研究者^[12]以 MobileNetv3 为基础对 SSD 骨干网络进行了结构优化，并采用 Soft-NMS 技术消除冗余检测结果，在自构建数据集上取得了 95.50% 的平均精度均值（mAP），同时将单幅图像的推理耗时降低至 30ms；KARTHIK 等学者^[13]创新性地融合了 Swin Transformer 架构与残差变形卷积网络，构建了双通道并行网络结构，显著改善了葡萄叶片病害智能分类的性能表现，分类准确率提升至 98.6%。

近年来，为进一步适应实际应用需求，研究者们纷纷致力于轻量化网络的设计。LI 等^[14]通过使用 ShuffleNetv2 重构骨干网络，并用 Add 操作替代 Concat，显著降低了计算量、参数量及模型体积，同时保持 98.8% 的准确率；XIAO 等^[15]结合通道剪枝与注意力机制，在减少模型体积至原有的约 21%（从 36.5M 减少到 7.8M）的同时，维持了 88.6% 的准确率。谭厚森等^[16]进一步优化了 YOLOv8 的部分模块，例如通过引入 Partial Convolution（PConv）和改良的空间金字塔池化（SPPF），在香梨检测数据集上使平均精度均值提高了 0.4–0.5 个百分点，检测速度分别提升了 34% 与 24.4%。

此外，LI 等^[17]还针对 YOLOv7-tiny 模型进行了结构优化，引入轻量特征提取模块与解耦头设计，并应用模型剪枝技术，实现了在保持良好检测性能的同时，将计算参数、模型复杂度和体积分别压缩到原网络的 37.8%、34.1% 和 40.7%。

1.3 论文主要内容

随着农业现代化的推进，番茄叶片病虫害问题依然对农作物产量与质量构成威胁。传统人工识别方式效率低、误差大，已难以满足精细化农业的需求。为提高识别效率与准确率，本文设计并实现了一种基于深度学习的番茄叶片害虫检测系统，旨在为农业病虫害防治提供智能化、自动化的技术支持。

本论文的主要研究内容包含以下几个方面：

1) 针对我国农业领域中番茄叶片病虫害识别效率低下的问题，选择番茄叶片病虫害作为研究目标，运用卷积神经网络进行特征提取的目标检测算法，以提高对小目标识别和复杂背景干扰的抵抗能力。选择 Ultralytics 团队研发的 YOLOv8 作为基础模型，该模型作为 YOLO 系列的第八代产品，保留了一阶段检测机制的优势，能在单次前向计算中同步预测目标位置与类别信息，在实时检测场景下保持了高效性与准确性。考虑边缘设备部署需求和 YOLOv8n 在本数据集上的良好表现，本文对原模型进行了两点轻量化改进，即利用 Ghost 模块特性减少模型冗余，并在此基础上剪枝，二次压缩模型。得到计算量和参数量显著降低的 Light-YOLOv8 模型。

2) 基于实际应用场景分析，构建了系统整体架构，将功能划分为识别搜索、记录管理和用户管理三大核心模块。并通过 UML 建模工具绘制了类图和时序图，明确了各组件间交互关系和数据流向，完成了功能设计与数据库设计。

3) 基于优化模型完成了系统开发。系统实现采用 B/S 架构的前后端分离模式，后端基于 Springboot 框架与 Mybatis-plus 构建，使用 Java 实现核心功能，并通过 Mysql

技术实现数据存储；前端则结合 Vue3.0 和 ElementUI 打造用户界面，同时利用 Flask 框架完成模型部署。各模块均通过功能性测试，运行良好。

1.4 论文组织结构

本文包含六个章节，具体内容构成如下：

第一章，绪论。讨论番茄叶片病虫害研究背景和价值，同时概述了农业病虫害识别算法的国内外发展现状，并简要介绍了论文的整体架构。

第二章，概述相关理论基础。主要阐释了系统采用的基于深度学习的目标检测算法，以及 Web 开发相关技术。

第三章，基于改进 YOLOv8n 的番茄叶片病虫害检测模型。引入了基于幽灵卷积模块和 L1 正则化筛选法剪枝的两种轻量化策略，以增强 YOLOv8n 的实时监测效能，改进后的模型被命名为 Light-YOLOv8，并通过对比实验和消融分析验证了该模型在准确率和效率方面的优越性。

第四章，番茄叶片病虫害识别的系统分析与设计。站在农业用户和数据管理者的视角，明确了功能需求和非功能性指标，按照用户角色将系统划分为识别搜索、记录管理和账户管理三大功能模块，并对各模块进行了详细设计。

第五章，番茄叶片病虫害识别系统的实现与测试。将第三章优化后的模型实际应用，采用 Vue 前端框架、Springboot 后端框架和 Flask 部署框架进行系统构建，对系统主要界面进行了展示。最后对所有功能模块实施了全面测试验证，充分确保系统的实际应用价值和可靠性。

第六章，结论和展望。通过对研究内容与实施结果的总结回顾，深入分析了当前系统存在的缺陷和限制，并针对多用户并发处理、数据管理优化以及算法改进等方面对未来工作提出展望。

2 相关理论与技术

2.1 基于深度学习的目标检测算法

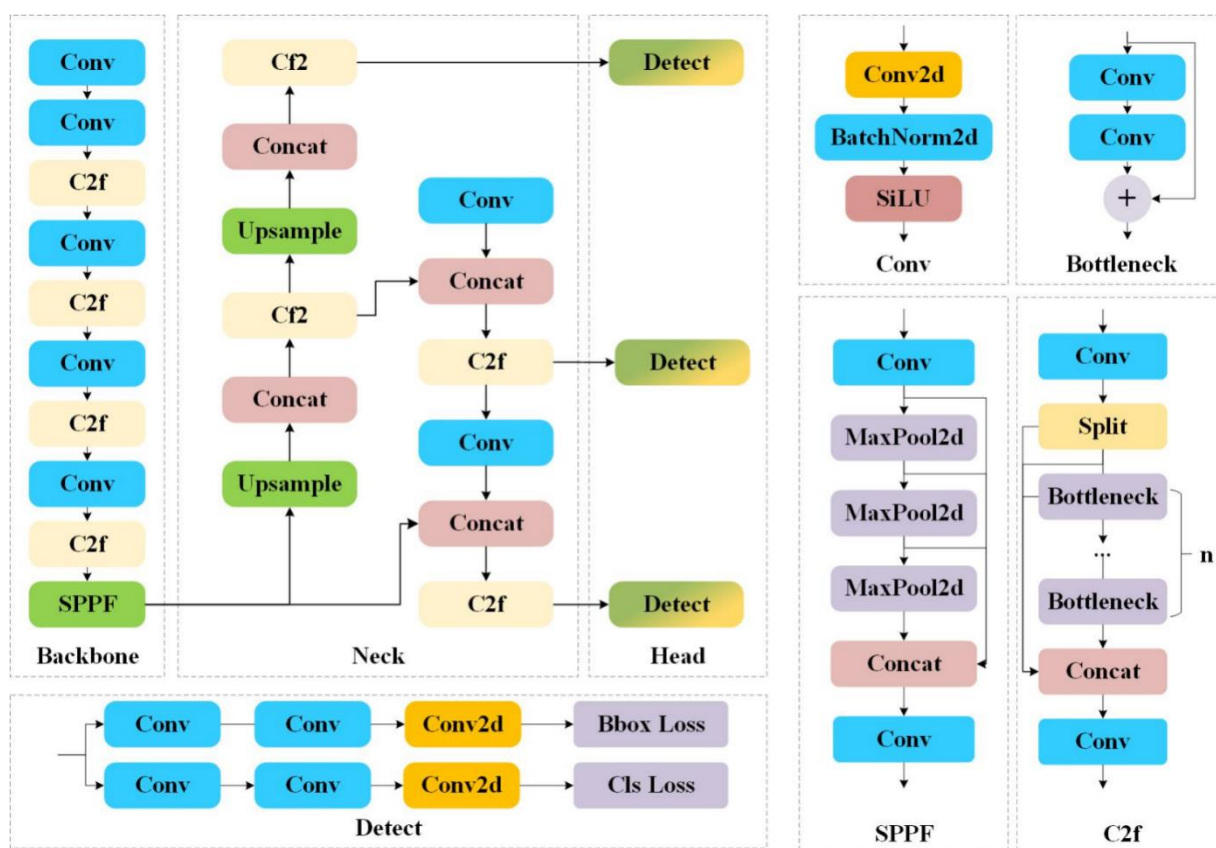
深度学习是机器学习的重要分支^[18]，利用卷积神经网络（CNN）可以自动进行病虫害的分类和定位。深度学习目标检测算法大致分为一阶段和双阶段两类。双阶段算法先生成候选框，再通过 CNN 提取特征分类；该方法的代表性算法包括 R-CNN^[19]及其改进版本，如 SPP-Net^[20]、Fast R-CNN^[21]、Faster R-CNN^[22]和 R-FCN^{错误!未找到引用源。}等都是在 R-CNN 的基础上进行了改进和优化。

而一阶段检测算法中图像直接被 CNN 处理，不进行候选框生成，既降低模型复杂度，又提升计算效率。常见的一阶段目标检测算法有 SSD^[10]和 YOLO^[11]模型系列。

YOLO 模型通过将输入图像划分为多个网格，每个网格负责预测该区域内的目标，这使得 YOLO 能够在单次前向传递中同时完成目标定位、候选框生成和分类任务，从而显著提高了检测速度。YOLO 系列还采用回归的方法进行边界框预测，并且通过卷积神经网络（CNN）直接从特征图中提取目标特征，避免了传统双阶段方法中需要候选框生成的复杂步骤。尽管精度和速度方面表现出色，但在处理小目标时的精度有所欠缺，尤其是在同一网格区域内存在多个目标时，可能会出现漏检现象。为了克服这些问题，后续版本对模型进行了多方面的改进，例如引入批量归一化（BN）提高训练稳定性、引入先验框机制改善目标定位、以及多尺度训练提升模型对不同尺寸目标的检测能力。同时，YOLO 系列还加强了特征融合和多层次信息的提取，使得模型在各种实际应用中表现更加出色。

YOLOv8^[24]是 YOLO 系列目标检测模型的第 8 个主要版本，由 Ultralytics 团队开发。它继承了 YOLO 系列模型的一阶段检测机制，根据参数量不同，YOLOv8 包含五个版本：YOLOv8n、YOLOv8s、YOLOv8m、YOLOv8l 及 YOLOv8x。YOLOv8n 架构包含三个核心部分：主干网络(Backbone)、特征融合网络(Neck)以及预测头(Head)，如图 2-1 所示。

Backbone 部分由 Conv，C2fckbone、和 SPPF 构成，负责提取图像多尺度特征。Neck 部分借鉴了 PAN 和 FPN 设计思想，将骨干网络获取的特征通过上采样进行细节特征增强，并通过下采样整合深层语义信息，从而实现不同尺度特征的高效融合。Head 部分采用解耦头(decoupled-head)结构，实现分类和回归的解耦处理，分别优化目标分类和定位精度，并引入 Task Aligned Assigner 标签分配策略，对分类分数和回归分数进行加权匹配，以提高正样本的匹配精度，此外采用无锚框架构，在降低计算复杂性的基础上增强了检测模型的泛化能力。

图 2-1 YOLOv8n 网络结构^[24]

2.2 系统开发 Web 技术

2.2.1 后端开发技术

1) SpringBoot 框架

2013 年起, Spring Boot 项目开始研发, 它是 Spring 家族中的新框架, 旨在简化 Spring 应用程序的创建和开发。Spring Boot 遵循“约定优先配置”原则, 避免了 XML 配置, 通过默认配置结合 Maven 等包管理工具, 自动配置数据库连接、Web 服务器、日志记录和安全性等。它还提供了大量的“起步依赖”(Starter Dependencies), 使开发者能够通过引入相应的依赖包, 快速集成所需的第三方框架, 生成 Spring 应用程序。此外, Spring Boot 集成了 Tomcat 等 Servlet 容器, 简化了项目部署, 只需将项目打包成 jar 或 war 文件, 即可在目标环境中运行。在开发过程中, 开发者可以通过 JavaConfig 类和注解等方式进行配置, 使代码更加简洁, 提升维护性。

2) MyBatis-plus

MyBatis-Plus 是基于 MyBatis 的增强框架, 旨在简化开发过程并提高效率。MyBatis 作为一款优秀的 ORM (对象关系映射) 框架, 在此基础上, MyBatis-Plus 提供了更多功能和特性, 使得开发者在进行数据库操作时更加轻松和高效。MyBatis-Plus 减少了 MyBatis 原有的复杂配置, 开发者只需少量配置即可使用大部分功能。它还提供了代码生成器和强大的通用接口, 能够帮助开发者快速完成基础功能的开发。

在 Web 开发中, MyBatis-Plus 通常与 Spring Boot 结合使用。基于 Spring Boot 的简洁配置方式与自动化配置功能, MyBatis-Plus 在数据持久层提供了强大且易用的支持, 使得两者结合能够显著提升开发效率。

3) Mysql

MySQL 作为一个被大量采用的免费开放关系型数据库管理系统, 其核心作用在于数据的存储与处理, 通过结构化查询语言进行各类操作。该系统具有高效能、适应性强、稳定性好等特点, 能够有效应对海量同时连接及复杂检索需求, 因此适合各种不同的应用环境。其高可用性与负载均衡能力使得系统在处理大量数据时性能出色, 尤其在样本查询或插入时表现优异。MySQL 还能够与多种编程语言和框架无缝集成, 方便开发者快速构建和部署应用。

4) Flask 框架

Flask 是一个用 Python 编写的轻量级 Web 应用框架, 首次发布于 2010 年。Flask 提供了最小化的核心功能, 允许开发者根据需求自由选择其他功能组件或扩展包。作为一个微框架, Flask 不强制要求使用特定的数据库、表单验证或认证机制, 开发者可以根据具体项目需求灵活选择, 极大地提高了开发效率和灵活性。

Flask 的主要特点是易于上手、灵活、扩展性强。它内置了基本的路由、模板引擎 (Jinja2) 以及 WSGI 兼容的开发服务器, 可以快速搭建 Web 应用原型。在开发中, Flask 支持多种扩展 (如 SQLAlchemy、Flask-Login 等), 使得它在处理更复杂的功能时也能保持轻便和高效。通过 Flask, 开发者可以快速构建小型 Web 应用或 RESTful API, 尤其适合用于开发原型、微服务和小型项目。此外, Flask 还拥有强大的社区支持和丰富的第三方扩展包, 使其能够满足各种开发需求。

2.2.2 前端开发技术

1) Vue 框架

Vue.js 是由尤雨溪于 2014 年开发并在 2015 年正式发布的渐进式前端开发框架, 具有轻量化特征。该框架致力于通过简洁的 API 接口和响应式数据绑定机制, 简化交互式用户界面的构建过程。Vue 专注于 MVVM (模型-视图-视图模型) 架构模式中的视图层实现, 采用数据驱动与组件化的设计理念。开发过程中可将复杂应用拆分为若干独立的小型组件, 通过自底向上的方式构建整体应用; 各组件均具备独立的模板、业务逻辑与样式定义, 从而实现代码的高度复用性, 减少模块间耦合, 简化开发流程。Vue3 版本中引入了组合式 API 特性, 将组件逻辑拆解为更细粒度的函数单元, 通过函数组合方式实现逻辑复用。Vue Router 作为 Vue 官方路由管理解决方案, 采用声明式配置方法定义路由规则, 建立路径与组件的映射关系。路由机制支持应用在不同页面间进行数据传递, 实现页面跳转与信息交互功能。Pinia 则是专为 Vue 设计的轻量级状态管理工具, 利用响应式数据来维护应用状态, 能够自动触发相关组件更新以保持界面数据一致性。Pinia 具备异步状态管理能力, 可有效处理异步操作、网络请求等复杂应用场景, 确保状态管理的稳定性与可靠性。

2) Element 组件

Element UI 作为一个开源的 Vue.js UI 框架，由饿了么前端团队研发，致力于为开发人员提供高品质、易于使用且外观精美的界面组件集合，主要面向桌面端应用程序的界面构建。该组件库包含了丰富的界面元素，覆盖表单控件、按钮、数据表格、对话框、页面布局等多种常用 UI 需求，能够协助开发者迅速搭建符合当代设计标准的应用程序。组件库采用响应式设计思路，保证在各种屏幕尺寸环境下都能实现良好的界面适配效果，从而优化用户使用体验。Element UI 在设计哲学上追求简约、实用与美观的统一，组件外观风格清爽且支持高度自定义配置。通过简化配置流程和灵活的组件调用机制，Element UI 显著降低了开发复杂度，让前端工程师能够将更多精力投入到核心业务逻辑的开发中，无需过度关注界面实现细节。同时，该框架配备了详尽的技术文档与使用示例，便于开发者快速掌握使用方法，并可根据项目具体需求进行组件样式定制，实现更佳的视觉呈现和功能表现。

2.3 本章小结

本章首先介绍了基于深度学习的目标检测算法，YOLOv8 作为一阶段算法，通过优化网络结构、引入新技术如解耦头、Anchor-free 模型等，显著提高了检测精度和速度；随后介绍了系统开发所使用的 Web 技术，后端使用了 SpringBoot+Flask 框架，简化了 Java 开发流程，同时使用 MySQL 数据库提供数据支持，前端技术采用 Vue+Element UI，提升了前端开发的效率和界面交互性。

3 基于改进 YOLOv8n 的番茄叶片病虫害检测模型

3.1 应用场景分析与问题描述

本系统面向农业领域中实际部署使用的病虫害检测场景，因此需要选择稳定、成熟与可维护的模型。YOLOv8 作为目前目标检测领域中应用广泛的主流模型之一，具备高度工程化的实现框架、完善的文档支持以及活跃的开源社区，适合部署于需要长期运行的农业图像识别系统中。马盼等^[25]出了一种基于 YOLOv8 的棉蚜图像识别算法，通过使用 YOLOv8l 模型进行训练，mAP50 达到 92.6%；石浩^[26]采用改进的 YOLOv8 模型应用于水稻籽粒图像分割，平均识别精度达到 89.2%；任梦真等^[27]针对蓝莓果实成熟度与嵌入式平台内存不足问题，构建 YOLOv8n-Ghost 模型，使得模型体积减小 17%，同时保持了较高的检测精度。YOLOv8n 是官方系列中体积最小、推理速度最快的模型，因其在对精度要求较低但需实时检测的场景中表现良好，因此被广泛应用。基于此，本文选取 YOLOv8n 作为基础检测模型。

在构建数据集并完成初步训练后，YOLOv8n 在本课题数据上取得了较高的识别精度，具备较强的特征提取与病虫害区域定位能力。经过多轮实验评估，YOLOv8n 在本任务中已基本满足识别精度的需求，但在实际部署中仍存在以下问题：

- 1) 在边缘设备、嵌入式系统或移动终端中部署模型时，对模型体积、计算资源消耗和推理速度有更高要求。尽管 YOLOv8n 已属轻量级模型，但其参数量和计算量（FLOPs）对于部分低功耗设备仍不够友好，存在进一步压缩空间。
- 2) 在农业巡检、实时识别等场景中，对模型的响应速度要求极高。为提高系统整体运行效率，减少识别延迟，有必要进一步对模型结构进行裁剪和精简，加快前端识别处理流程。
- 3) YOLOv8n 虽为通用模型，在实际任务中仍存在部分模块对特定场景的适应性不强或特征冗余的情况。通过结构分析和剪枝策略，可识别并移除部分对本任务贡献较小的通道或层，达到压缩模型的目的。

3.2 基于 Ghost 模块的卷积轻量化改进

YOLOv8 网络的主干部分采用了多种标准卷积组件，这些组件主要负责抽取图像深度特征信息，以提升模型对目标的感知能力。常规卷积运算过程中往往产生大量重复的特征表示，尽管这类冗余信息为网络提升了一定的特征表达能力，其生成过程却伴随着高昂的参数与运算负担，最终造成网络结构庞大、推理速度缓慢等制约性问题。

GhostNet 网络由 Han 等人^[28]于 2020 年提出，旨在解决卷积神经网络在嵌入式设备上部署时的高计算成本问题。研究人员指出，CNN 强大的特征提取能力在很大程度上依赖于存在一定冗余的信息。因此，GhostNet 通过引入一系列计算代价低廉的线性变

换（cheap linear operations），从原始特征中生成称为 Ghost 特征图（feature maps）的冗余信息，以较低的计算开销提取有效特征，从而提升模型的整体效率。

GhostNet 的提出者 Han 等人通过在 ImageNet 分类任务中的大量实验证明：Ghost 模块能够有效生成冗余特征，在大幅度减少计算量和参数数量的同时，保持模型的分性能，其在轻量化网络领域已被广泛应用于分类、检测和分割等多个任务中，是目前较为主流的轻量化结构之一。Yang 等^[29]在改进的 YOLOv5 模型的基础上，在主干网络中引入 C3Ghost 模块使模型轻量化，精确度较原网络模型 mAP 提升了 2.4%；景亮等^[30]在 GhostNet 中引入注意力机制 CA 对空间信息和通道信息进行融合，提升目标关注度，保持精度的同时模型参数下降。

为此，本文引入 GhostNet 网络的设计思想，对 YOLOv8 的 Backbone 结构进行了轻量化改进。GhostNet 网络由其核心组件 Ghost 模块（Ghost Module）堆叠而成。

GhostModule 以低参数量产生丰富特征信息的特点，使它能够替代网络结构中的标准卷积。如图 3-1 和图 3-2 所示，常规卷积直接生成输出特征图，而 GhostModule 先通过卷积提取少量原始特征，再用简单线性变换进行分组处理，生成冗余特征信息，有效增强模型表达能力，实现轻量高效的特性。

GhostModule 将原始特征与廉价操作生成的冗余特征由残差连接拼接后输出，确保结果与标准卷积相当。

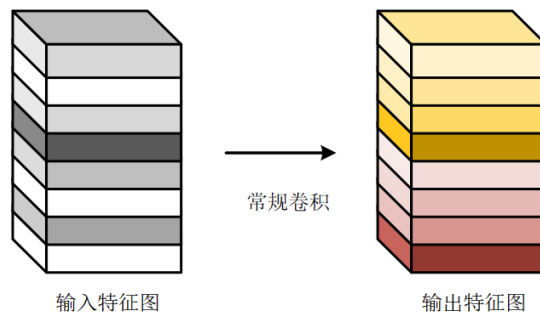


图 3-1 特征在标准卷积下生成

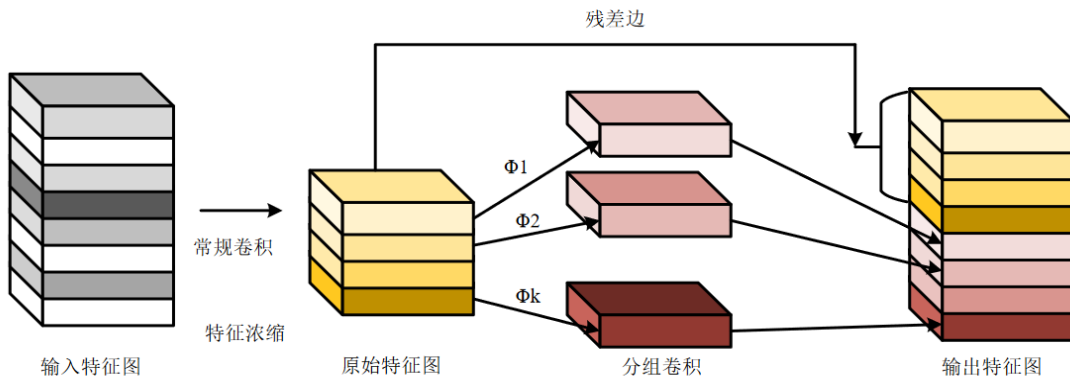


图 3-2 特征经过 Ghost 模块生成

设输入特征图为 $X \in R^{c \times h \times w}$ ，卷积核 $f \in R^{c \times k \times k \times n}$ ，输出特征图 $Y \in R^{h' \times w' \times n}$ ，常

规卷积操作计算公式为：

$$Y = X * f + b \quad (3-1)$$

其中, c 、 h 、 w 分别为输入通道数、高度、宽度, n 为输出通道数; $*$ 表示卷积操作; k 为卷积核大小; b 为偏置项。常规卷积的计算量 $FLOPs_{conv}$ 和网络参数量 $Parameters_{conv}$ 由下式计算:

$$FLOPs_{conv} = n \times h' \times w' \times c \times k \times k \quad (3-2)$$

$$Parameters_{conv} = n \times c \times k \times k \quad (3-3)$$

Ghost 模块进行卷积的计算公式如下:

$$Y' = X * f' \quad (3-4)$$

其中, m 个通道的初始特征图 $Y' \in R^{m \times h \times w}$, 其尺寸小于原始特征图。卷积核 $f' \in R^{c \times m \times k \times k}$ 大小为 $k \times k$, 且 $m \leq n$ 。该过程不含偏置项, 其他超参数与标准卷积一致, 以确保输出特征图的空间维度不变。

为实现输出特征图通道数达到 n , Ghost Module 对初始特征图 $Y' \in R^{m \times h \times w}$ 实施分组卷积操作, 产生 s 组冗余特征, 其计算表达式如下:

$$y_{ij} = \phi_{i,j}(y'_i), \forall i = 1, \dots, m, j = 1, \dots, s \quad (3-5)$$

其中, y_i 代表原始特征图 Y 的第 i 个分段, 用于产生 s 个冗余特征。其中, $\phi_{i,j}$ 表示对 y_i 执行的第 j 次线性变换 (不包括最后一次), 生成第 j 个冗余特征 $y_{i,j}$, 而最后的 $\phi_{i,s}$ 则用于恒等映射。由于恒等映射的计算开销远低于标准卷积, 这一设计有效降低了模型的计算负担。最终, Ghost Module 能够输出通道数为 $n=m \times s$ 的特征图, 其中原始特征图的通道数满足 $m=n/s$ 。

在分组卷积阶段, Ghost Module 共执行 $m \times (s-1) = (n/s) \times (s-1)$ 次线性操作, 其平均卷积核尺寸为 $d \times d$ (与 $k \times k$ 相近), 初始特征图和冗余特征图通过残差连接被拼接起来。此过程概括为三步: 生成原始特征图、生成冗余特征图和拼接, 过程所需的计算量 $FLOPs_{ghost}$ 和参数量 $Parameters_{ghost}$ 由下式计算:

$$FLOPs_{ghost} = \frac{n}{s} \times h' \times w' \times c \times k \times k + (s-1) \times \frac{n}{s} \times h' \times w' \times d \times d \quad (3-6)$$

$$Parameters_{ghost} = \frac{n}{s} \times c \times k \times k + (s-1) \times \frac{n}{s} \times d \times d \quad (3-7)$$

这里, 由于 $s \ll c$, 通过上述公式可以看出, 采用 Ghost Module 模块后, 网络的计算量和参数量相比传统卷积操作均显著降低, 降幅约为 s 倍, 计算如下:

$$\frac{FLOPs_{conv}}{FLOPs_{ghost}} = \frac{c \times k \times k}{\frac{1}{s} \times c \times k \times k + \frac{s-1}{s} \times d \times d} \approx \frac{s \times c}{s+c-1} \approx s \quad (3-8)$$

$$\frac{Parameters_{conv}}{Parameters_{ghost}} \approx \frac{s \times c}{s+c-1} \approx s \quad (3-9)$$

图 3-3 展示了 GhostBottleneck 的详细结构设计, 步长为 1 时直接转换特征; 为 2

时则通过下采样和深度可分离卷积实现空间压缩并创建快捷连接。这种模块化设计既保留了特征提取能力，又显著降低了计算复杂度。

相比 C3 模块，C2f 结构中引入了更密集的 Bottleneck 连接，虽然梯度信息更丰富，但计算复杂度也略有增加。而 C3 模块在上下文信息的捕捉上的表现更为优异。为在模型精度与效率之间取得更好的平衡，本文选择将 Backbone 层中的 C2f 替换为 C3Ghost 模块。

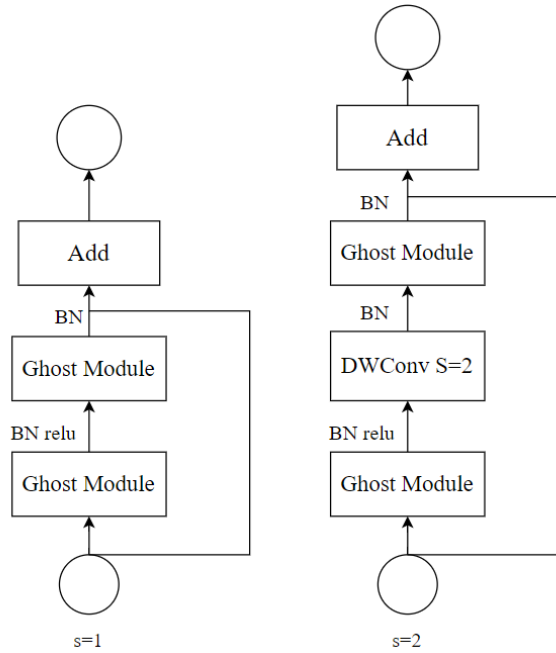


图 3-3 Ghost Bottleneck 瓶颈模块的结构图

如图 3-4、图 3-5 所示，C3Ghost 模块由 3 个标准卷积和 n 个 GhostBottleneck 组成，其中每个 GhostBottleneck 结构由两个 Ghost 模块堆叠而成。这种设计使得网络在保持特征提取能力的同时显著降低计算成本，从而实现模型轻量化并提升性能。该结构不仅继承了残差连接与跨阶段特征融合的优势，还通过引入 GhostConv 操作，有效压缩模型体积，提高推理速度。

综上，本文最终对原 YOLOv8n 模型的 Backbone 网络结构进行了如下改进：将其中的标准卷积模块统一替换为 GhostConv 模块，以减少参数量；同时将 C2f 模块替换为轻量化的 C3Ghost 模块。经过骨干网络优化后，图像处理流水线呈现如下特点：系统接收 $640 \times 640 \times 3$ 尺寸的原始输入，首轮经 CBS 结构（集成卷积层、批归一化与 SiLU 激活器）处理，生成 $320 \times 320 \times 64$ 规格的初级特征。继而通过两组 Ghost 卷积 CBS 单元与一组 C3Ghost 架构，将数据依序转化为 $160 \times 160 \times 128$ 、 $80 \times 80 \times 128$ 维度表示，同时保留空间分辨率。特征随后流经 Ghost 卷积 CBS 与 C3Ghost 结构链，实现 $40 \times 40 \times 256$ 双重表征；后续环节产生 $20 \times 20 \times 512$ 双通道特征映射；末端环节由 Ghost 卷积 CBS 变换出高阶语义信息，最终传递至 SPPF 单元进行跨尺度特征融合，完成特征提取全过程。

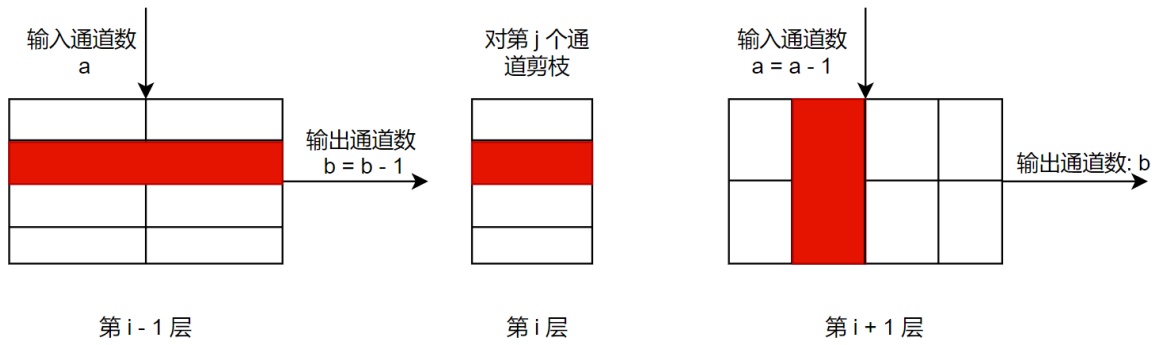


图 3-6 卷积核剪枝的过程图

其中，设 i 为卷积层的编号，图 4.4 中的红色区域代表需要被剪枝的卷积核结构。当对第 i 层实施剪枝时，其前后相邻卷积层的对应区域也会同步进行裁剪，以保证网络结构的连贯性。

在网络前向传播过程中，第 i 层接收尺寸为 $R^{n_i \times h_i \times w_i}$ 的输入特征图 x_i ，该层的卷积核由 n_i 个二维滤波器 $K \in R^{k \times k}$ 组成，整体可表示为三维张量 $F_i \in R^{n_i \times n_{i+1} \times k \times k}$ 。由此，第 i 层到第 $i+1$ 层的总计算量 $FLOPS_{sum}$ 可表示为：

$$FLOPS_{sum} = n_i \times n_{i+1} \times k \times k \times h_{i+1} \times w_{i+1} \quad (3-10)$$

其中，输入层特征图 x_i 的参数包括通道维度 n_i 及空间维度 h_i （高）和 w_i （宽）。该特征经卷积运算处理后，转化为形状为 $R^{n_{i+1} \times h_{i+1} \times w_{i+1}}$ 的新特征图 x_{i+1} ，随后被传递至网络的第 $i+1$ 层进行后续计算。

基于 L1 范数准则，本章方法能够识别出对病虫害特征提取贡献较低的卷积核。假设第 i 层中共有 m 个冗余卷积核，剪枝时需同时移除第 i 层的卷积核 $F_{i,j}$ 及其在第 $i+1$ 层对应的特征图通道 $x_{i+1,j}$ 。这一操作可减少的计算量 $FLOPS_{one}$ 由以下公式计算：

$$FLOPS_{one} = n_i \times k \times k \times h_{i+1} \times w_{i+1} \quad (3-11)$$

根据公式(4-10)和(4-11)，当剪除第 i 层第 j 个卷积核 $F_{i,j}$ 时，其减少的计算量占第 i 层到第 $i+1$ 层总计算量的比例为 $1/n_{i+1}$ ，具体推导如下：

$$\frac{FLOPS_{one}}{FLOPS_{sum}} = \frac{n_i \times k \times k \times h_{i+1} \times w_{i+1}}{n_i \times n_{i+1} \times k \times k \times h_{i+1} \times w_{i+1}} = \frac{1}{n_{i+1}} \quad (3-12)$$

若剪除全部 m 个冗余卷积核，则减少的计算量记为 $FLOPS_m$ ，计算公式为：

$$FLOPS_m = m \times n_i \times k \times k \times h_{i+1} \times w_{i+1} \quad (3-13)$$

根据公式(3-10)和(3-13)的分析结果， $FLOPS_m$ 在整体计算量中所占比例为 m/n_{i+1} 。这一结果表明，本文采用的轻量策略能够显著降低模型的计算复杂度，从而有效提升系统运行效率。

在方法设计上，本文采用 L1 范数准则来评估卷积核的重要性。具体而言，通过计算第 i 个卷积层中第 j 个卷积核 $F_{i,j}$ 的权重绝对值之和 $\sum |F_{i,j}|$ ，可以量化该卷积核对特征提取的贡献程度，它用 L1 范数准则可表示为 $\sum \|F_{i,j}\|$ 。首先，计算卷积核 L1 范数得分

当压缩率达到预设目标后，剪枝过程终止，随即开始微调重训练阶段，旨在恢复因网络结构变化而导致的精度下降问题。实验结果表明，该方法不仅能够维持较高的识别精度，还能显著提升模型的运行效率。这为后续将模型集成到实际系统中提供了便利，同时也增强了模型在各类轻量级设备上的部署能力。此优化方案既保证了模型在系统中的实时性能，又具有部署到边缘设备的潜力。

实际应用层面上，本文将 Ghost 模块架构与剪枝技术进行了结合，对原有的 YOLOv8n 模型进行了轻量化改进，最终构建了 Light-YOLOv8 模型。如错误!未找到引用源。所示，该模型的检测流程中，蓝色框标注部分即为采用轻量化改进的关键环节。

3.4 实验设计和结果分析

本节将展示两种 YOLOv8 轻量化方法实验结果，对比原始模型与优化模型的识别精度、模型体积及推理速度。3.2 节采用 GhostNet 替换主干网络减少参数量；3.3 节引入结构剪枝精简模型。通过对比三种模型性能，分析不同轻量化策略在病虫害图像识别中的效果。

3.4.1 实验数据集及实验环境

1) 番茄叶片病虫害数据集

本数据集包含 10 类番茄叶片病虫害图片，共 15158 张，其中，番茄细菌斑病 2127 张，番茄早疫病 1000 张，番茄叶霉病 952 张，番茄斑点萎蔫病 1676 张，番茄晚疫病 1907 张，番茄灰霉病 1771 张，番茄曲叶病毒病 373 张，番茄斑点枯萎病 1404 张，番茄黄化曲叶病毒病 2357 张，番茄健康叶片 1591 张，研究数据来自于公开的 Plant Village 工程^[32]中的 Tomato 数据集以及互联网收集，收集到的图片使用 Labelme 标注工具进行标注。图 3-9 展示了该数据集中若干典型番茄叶片病虫害样本图像。



图 3-9 数据集部分样本

2) 数据增强

因番茄病虫害样本数量有限且分布不均（如细菌斑病比斑点枯萎病多 723 张），本文采用翻转、随机裁剪和模糊等数据增强方法扩充训练集。

将预处理后的数据划分为训练、验证与测试三个子集，其比例为 6: 2: 2。数据增强后，各类别样本量分布如下：番茄细菌斑病 2127 张，番茄早疫病 1500 张，番茄叶霉病 1500 张，番茄斑点萎蔫病 1676 张，番茄晚疫病 1907 张，番茄灰霉病 1771 张，番茄曲叶病毒病 1500 张，番茄斑点枯萎病 1404 张，番茄黄化曲叶病毒病 2357 张，番茄健康叶片 1591 张。

3) 实验硬件环境

本文的实验环境配置详见表 3-1。

表 3-1 实验环境

设备/软件类别	参数
内存容量	32GB
图像处理单元	NVIDIA RTX4090 24GB
存储空间	1TB
操作系统环境	Windows 10
深度学习框架	Pytorch-GPU
编程语言	Python3.8

为保证实验对比的公平性，各检测模型训练过程中均使用相同的超参数配置：每批处理 32 个样本，总计训练 100 个周期，学习率初值设为 0.005，优化方法采用 Adam 算法。

3.4.2 实验结果评价指标

本章节采用的评估标准包括：精确率（Precision）、召回率（Recall）、平均精度均值（mAP）、浮点运算量（FLOPs）以及每秒处理帧数（FPS）。其中前三项评价指标的计算方法如下所示：

$$P_r = \frac{TP}{TP+FP} \quad (3-15)$$

$$R_e = \frac{TP}{TP+FN} \quad (3-16)$$

$$mAP = \frac{1}{M} \sum_{j=1}^M Ap(j) \quad (3-17)$$

在上述公式中， TP 、 FP 、 FN 分别代表：真正例（被正确识别的目标类别样本数）、假正例（错误归类为目标类别的样本数）以及假负例（未被识别但实际属于目标类别的样本数）。系统对第 j 类的平均精度值记为 $Ap(j)$ ，番茄病虫害总类别数记为 M 。精确率 P_r 主要评估模型识别特定番茄病害的准确性，而召回率 R_e 则衡量模型对特定类别样本的发现能力。模型计算出所有番茄病虫害类别平均精度的均值记为 mAP 。模型前向推理过程中需执行的浮点计算总量为 $FLOPs$ ，模型每秒能处理的番茄病虫害图像数量为

FPS（帧率）。

3.4.3 对比实验分析

为评估本文提出的 Light-YOLOv8n 与现有模型的性能差异，选取了 YOLOv5 系列、YOLOv7 系列、Faster-RCNN 以及 SSD 模型与本文模型进行对比，所有模型均使用相同数据集与硬件进行训练与测试。实验结果详见表 3-2。

表 3-2 不同模型对比实验结果

模型 Model	准确率 Precision/%	召回率 Recall/%	平均精度均值 mAP0.5/%	计算量 FLOPs/G	FPS / (帧·s ⁻¹)
YOLOv5s	85.1	83.7	85.2	15.8	70.1
YOLOv7	89.1	87.4	89.8	106.5	36.9
Faster-RCNN	90.5	88.6	90.1	121.4	55.1
SSD	88.6	92.3	93.3	137.8	24.3
YOLOv8n	94.5	93.6	95.5	8.2	76.9
Light-YOLOv8n	93.4	93.2	92.4	3.4	103.7

实验结果表明，与 Faster R-CNN 和 SSD 相比，本文模型在浮点计算量（FLOPs）方面显著降低，分别减少了 117.9、134.3G，检测速度分别提高了 40.6、71.4 帧/s。

与 YOLOv5-Nano、YOLOv7 和 YOLOv8n 相比，在保持精度基本一致的前提下，本文算法的 FLOPs 分别降低了 1.0、103.0、4.6G，同时 FPS 提高了 33.6、66.8、26.8 帧/s。

综上所述，本文提出的 Light-YOLOv8n 算法相比所选的六种模型，在运行效率与识别准确率方面取得了相对较好的检测效果，初步验证了其在番茄叶片病虫害识别任务中的可行性。与此同时，该模型在参数量、浮点计算量以及模型体积等方面也展现出一定的优势，具备在资源受限设备上实现部署的潜力。

3.4.4 消融实验分析

为评估本章所提出两种轻量化策略在番茄叶片病虫害目标检测任务中的优化效果，本节设计了消融实验，分别将两种方法引入原始模型中进行对比分析。当两种方法同时应用时，模型即为本文最终提出的 Light-YOLOv8n。实验结果见表 3-3。为评价本章节提出的双重轻量化技术在番茄病虫害检测场景中的实际效能，本节构建了消融实验方案，逐步将这两种优化手段应用至基准模型中进行系统性比较。本文设计的 Light-YOLOv8 模型，即是两项技术被同时整合应用后的结果。相关实验数据结果已整理于表 3-4 中。

表 3-3 模型轻量化优化前后各指标对比

模型 Model	准确率 Precision/%	召回率 Recall/%	平均精度均值 mAP0.5/%	浮点计算量 FLOPs/G	FPS / (帧·s ⁻¹)
YOLOv8n	94.5	93.3	95.5	8.2	76.9
YOLOv8n+Ghost	93.8	92.9	93.1	6.3	95.1
Light-YOLOv8n	93.4	93.2	92.4	3.4	103.7

YOLOv8+Ghost 模型相比原模型 mAP 下降 0.7%，但 FPS 提升 18.2，浮点计算量减少 1.9G。

在此基础上采用模型剪枝，得到的 Light-YOLOv8n 模型较 YOLOv8n+Ghost 的 mAP 下降 0.7%，FLOP 降低 46%，FPS 提升 8.6%。与原始 YOLOv8n 相比，mAP 下降 1.1%，FLOPs 减少 56.8%，FPS 提升 26.8%。

综上所述，本文提出的两种轻量化策略在提高计算效率的同时保持了较高的检测精度，证明了其在番茄叶片病虫害检测任务中的有效性。

3.5 本章小结

本章围绕 YOLOv8n 模型的轻量化改进展开，提出并验证了两种有效的优化方法。首先，通过引入 GhostConv 模块替代 Backbone 部分原有的标准卷积，有效降低了模型参数数量和计算复杂度，同时保持了较高的识别精度。随后，设计了基于模型剪枝的轻量化策略，对卷积层中冗余或贡献较低的卷积核进行裁剪，并结合重新训练的方式恢复模型性能。在此基础上，综合应用两种方法构建了改进模型 Light-YOLOv8。为验证所提方法的有效性，本章设计并开展了多组实验，包括与主流大参数模型、YOLO 系列模型及 YOLOv8n 的性能对比，结果显示 Light-YOLOv8 模型在保持识别准确率的同时显著提升了推理速度与运行效率。此外，消融实验进一步证实两项轻量化策略均对模型性能优化有积极作用。综上，Light-YOLOv8 模型在准确率与轻量化之间实现了良好平衡，为后续部署与应用提供了更优方案。

4 番茄叶片病虫害识别系统的需求与设计

4.1 系统需求分析

本系统旨在积极响应国家智慧农业的发展战略，助力农业信息化水平的提升与农业生产方式的转型升级。当前，辣椒作为重要的经济作物，在种植过程中常面临多种病虫害的威胁。然而，多数农户仍主要依赖个人经验和肉眼判断进行病害识别，这种方式不仅效率较低，还存在较大的主观性和误判风险。为解决上述问题，系统引入深度学习算法实现图像自动识别，结合前后端集成的 Web 技术，开发出一套面向辣椒病虫害的智能识别与管理平台。

该系统面向一线农业生产者，尤其是广大辣椒种植户，通过图像识别功能帮助用户快速获取病虫害类别与相应的防治方案，辅助其科学决策和及时应对，从而减少损失、提高产量。同时，系统还具备百科知识查询、识别记录管理、用户账户维护等多种实用功能，力求为农业生产实践提供准确、高效、易用的技术支持。

根据以上需求分析，可以梳理出本系统的功能应当包括：识别搜索功能、识别记录管理功能、用户信息管理功能。

4.1.1 系统功能性需求分析

该系统划分为两种用户类型：普通用户与超级管理员，分别承担不同功能权限。

1) 识别搜索功能

图片识别：用户上传图片，点击识别，系统返回识别结果并自动归档记录。

视频识别：用户上传视频，系统返回带有类别标签框的视频并自动保存记录。

摄像头识别：开启摄像头实时返回识别结果，结果显示在摄像画面中。

2) 识别记录管理功能

图识别记录查看：用户在此页面查看图片、视频以及摄像头识别历史记录信息，包括识别时间、虫害类别、识别用时，用户可选择删除记录。视频记录查看时可点击查看原始视频。

识别记录标记：作为管理员可对识别记录进行标记，状态标为“忽略”的识别记录将不再显示在列表中，状态为“保留”的记录被视为优质图片数据，有待导出。

识别记录批量导出：管理员有权对已保留的识别记录进行批量导出，可选择导出全部保留记录、导出最近三天的记录或者导出新保留的记录。

3) 用户信息管理功能

注册：新用户通过填写必要信息完成账户创建。

登录：已注册用户输入账户凭证登录系统，系统生成会话令牌维护登录状态。

个人信息查询：用户可查看个人档案，包括姓名、电话、邮箱等基础信息。

信息更新：用户可编辑和更新个人档案信息。

用户管理：管理员有权对普通用户的账号信息进行查看、添加与删除。

基于上述角色功能需求分析，图 4-1 为本系统的用例图设计。

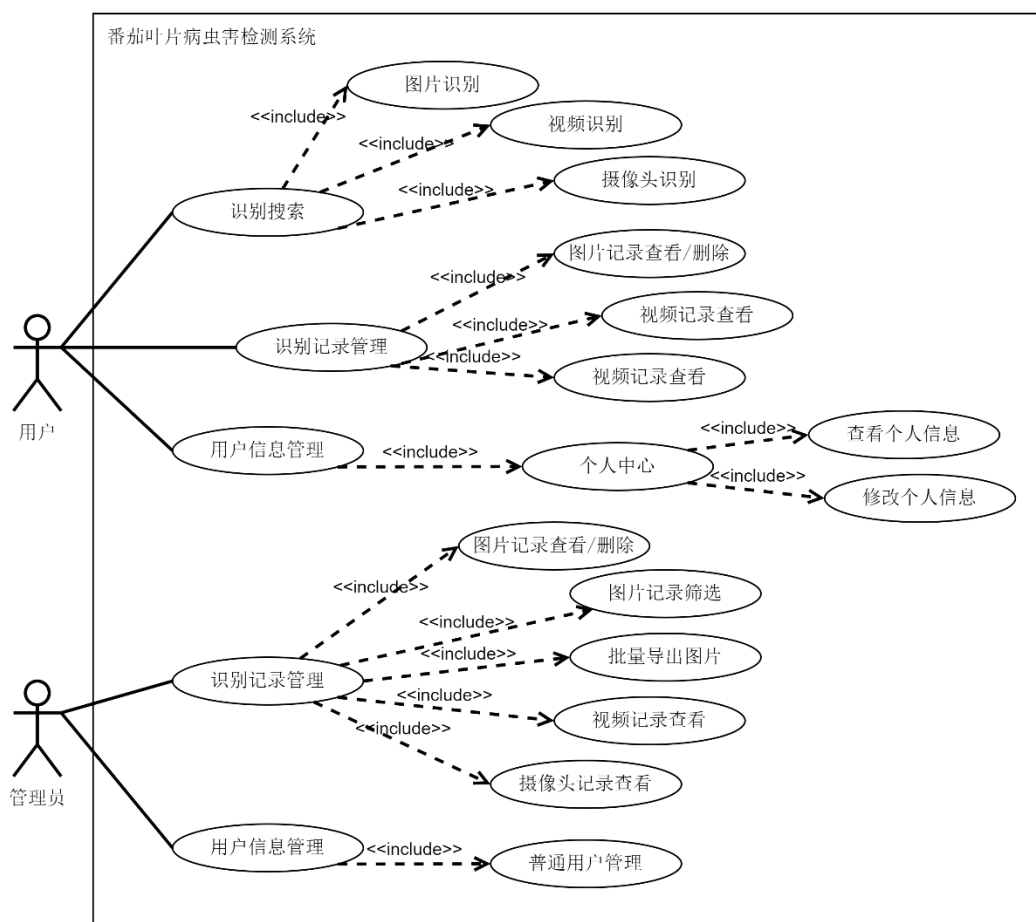


图 4-1 番茄叶片病虫害识别系统用例关系图

4.1.2 系统非功能性需求分析

除功能性需求外，本系统具有下列非功能性指标要求：

1) 响应性能要求

考虑到用户多为农业从业者，日常操作习惯偏向直观高效，系统需具备良好的响应速度。在网络良好的条件下，系统应支持至少 50 个并发用户同时上传与识别图像，系统对上传图片的识别响应时间应控制在 2 秒以内，保证用户能快速获取识别结果与防治建议，不影响现场处理进度。尤其在田间地头使用移动设备操作时，系统页面加载与功能切换的响应时间应控制在 1 秒以内。

2) 识别准确率要求

识别结果的准确性直接影响农户的防治决策。系统基于 YOLOv8 目标检测模型进行训练优化，常见病虫害（如疫病、蚜虫、病毒病等）的单类识别精度应高于 90%。此外，对于模糊、光照不均等劣质图像，系统需具备基本的鲁棒性，降低误判率。

3) 数据安全与隐私保护

农户上传的图像中可能包含种植区域、农场设备等敏感信息，系统需对上传图像

与识别结果采用 HTTPS 加密传输并进行本地存储加密，确保用户数据在传输与存储过程中的安全性。同时，对账号权限进行精细化控制，农户仅可访问本人数据，管理员拥有数据管理权限，保障数据不被滥用。

4.2 系统概要设计

4.2.1 系统整体体系结构

系统采用 MVC 分层架构设计理念，构建了表示层、业务逻辑层与数据持久层三个层次，系统整体架构设计见图 4-2。

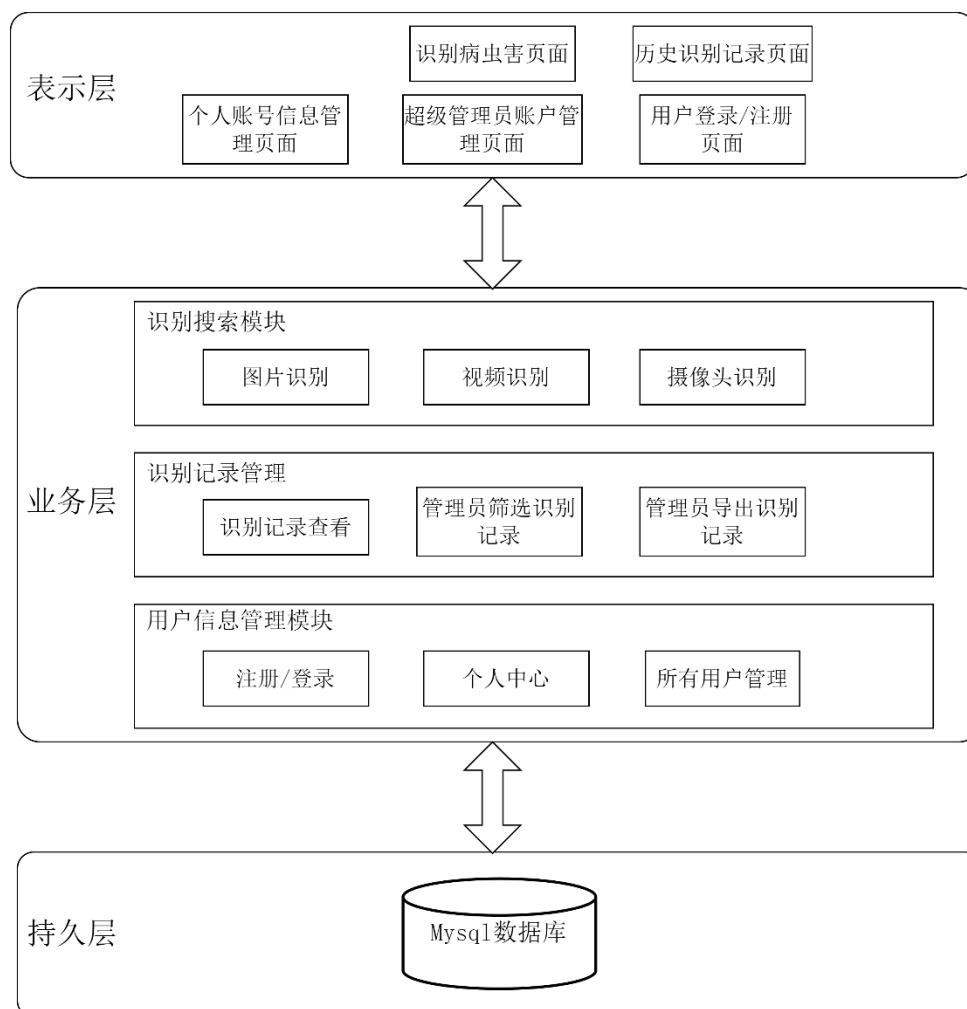


图 4-2 系统整体架构图

表示层负责为系统的两类用户（普通用户与超级管理员）提供直观、易用的图形化交互界面。普通用户可见的功能有：注册登录、识别检测、结果展示存储、历史数据管理等。用户可在识别模块中上传媒体文件获取识别结果，结果可保存至个人历史记录中，便于随时查看、下载或删除。此外，用户还可在“账户管理”页面修改密码或更新账户信息。对于超级管理员，系统提供了账户管理和图片数据筛选功能。管理员有权限对用户信息做出修改；对用户的识别记录，能够管理用户历史识别记录，包括对有效数

据图像的分类、筛选整理与批量导出，为后续模型训练数据集扩充提供支持。

业务层作为系统逻辑处理的核心，负责接收来自前端的请求并执行相应操作，完成后将处理结果返回表示层进行展示。该层主要涵盖：①识别服务模块：调用部署好的辣椒病虫害识别模型，接收用户上传图像，进行图像预处理后送入模型进行推理，并返回识别结果与置信度。②账户与管理模块：负责处理用户注册流程、身份验证、权限控制以及个人信息更新等操作。③数据管理模块：实现用户历史识别记录的存储、查询、删除及管理员对记录的筛选标记、批量下载功能，为模型提供新数据的积累渠道。

持久层负责将系统中关键的数据内容进行结构化存储和管理，确保数据安全、稳定、可持续地支持业务运行。本系统使用 MySQL 数据库进行数据持久化，主要存储内容包括：用户信息、识别记录、病虫害百科、系统操作日志（可选）。

4.2.2 系统功能模块设计

根据以上对基于音频内容检索的网络音乐侵权判别系统的需求分析，可以把该系统划分为 3 大功能模块：识别搜索模块、数据管理模块、账户管理模块，如错误!未找到引用源。所示。

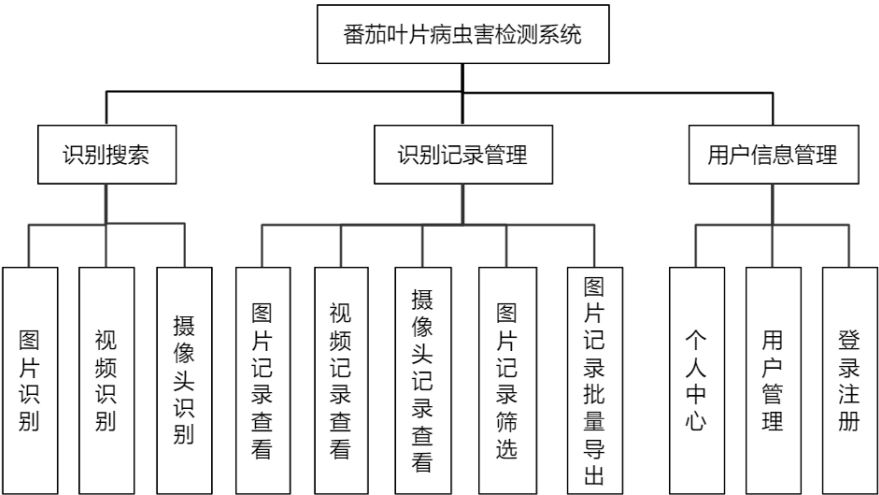


图 4-3 系统功能模块图

识别搜索模块负责病虫害识别与交互功能，用户可通过文件上传或视频上传提交本地图片或视频片段进行分析，也可实时开启摄像头拍摄叶片图像进行识别，系统调用第 3 章设计的模型完成病虫害检测后提供详细的虫害百科信息供用户查阅，所有识别结果可保存至个人记录供日后查看。

用户管理模块处理系统权限与账户相关操作，普通用户通过登录注册功能访问系统，在个人中心可修改基本资料、更新密码或提交意见反馈，管理员则拥有更高权限，可进行账户管理如新增或删除用户。

记录管理模块用于存储和管理历史识别数据，普通用户可查看、删除个人识别记录，并按时间或病虫害类型进行检索。管理员能查看所有用户记录并进行批量操作，同时通过标记操作，标记出有效数据并批量导出，可用于归档至训练数据集以持续优化

模型性能。

4.3 系统详细设计

4.3.1 识别搜索模块设计

本模块主要的核心逻辑在 FileController 和 PredictController，FileController 实现了 upload（）接口。Flask 端用于部署和调用模型。

图 4-4 显示了该模块的类图。

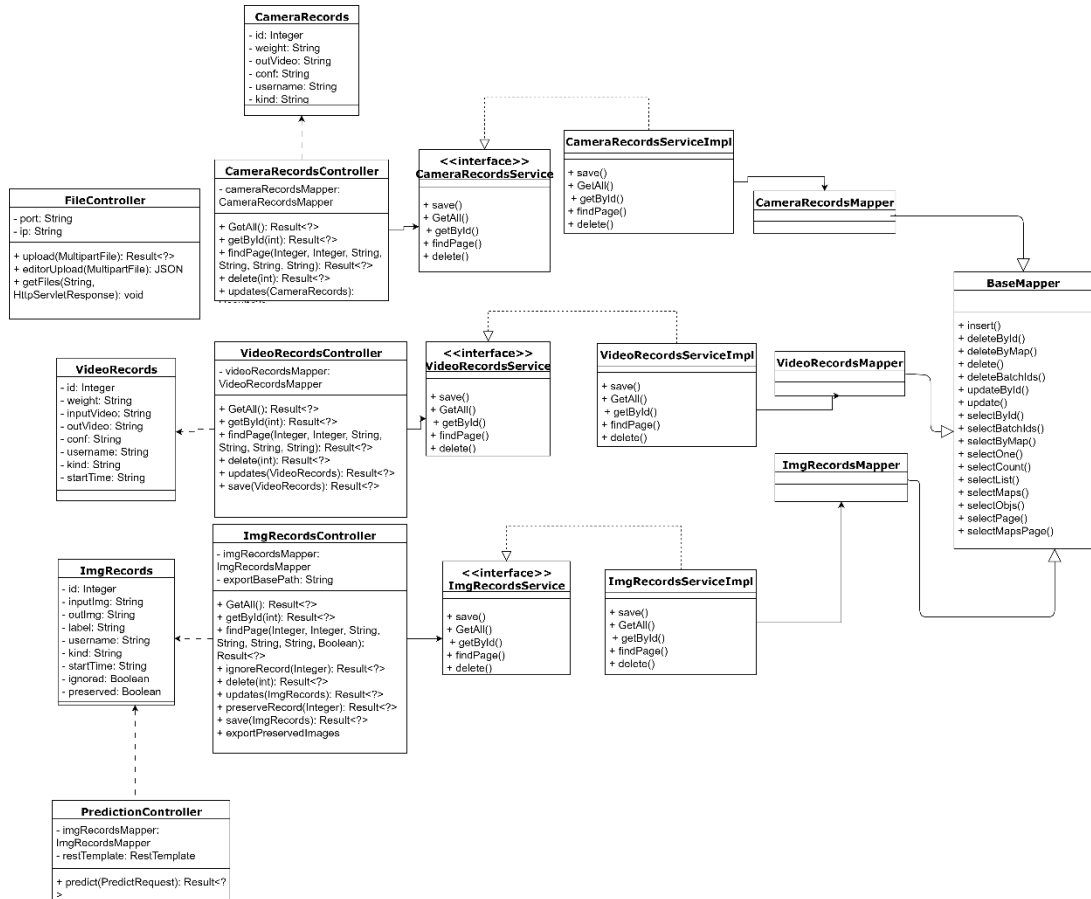


图 4-4 识别搜索模块类图

在数据访问层，ImgRecordsMapper 负责处理与图片识别记录表相关的数据库操作，包括插入、更新、查询等操作。VideoRecordsMapper 负责处理与视频识别记录表相关的数据库操作，CameraRecordsMapper 则负责处理与摄像头识别记录表相关的数据库操作。在控制器层面，PredictionController 作为预测功能的核心控制器，负责处理图片预测请求，调用外部 Flask API 进行模型推理，并将预测结果保存到数据库中。该控制器还提供了获取模型文件名的功能接口。FileController 专门负责文件管理相关的业务处理，包括文件上传、富文本编辑器文件上传以及文件下载等功能的实现。ImgRecordsController 负责图片识别记录的管理业务，提供了分页查询、记录忽略、记录保留、批量导出保留图片等功能的具体实现。VideoRecordsController 则负责视频识别记录的管理，提供了

记录的增删改查和分页查询等业务逻辑处理。**CameraRecordsController** 负责摄像头识别记录的管理,同样提供了完整的 **CRUD** 操作和分页查询功能。在实体层面, **ImgRecords** 定义了图片识别记录的数据结构,包含了输入图片、输出图片、识别标签、置信度、用户信息等完整的识别记录信息。**VideoRecords** 定义了视频识别记录的数据结构, **CameraRecords** 则定义了摄像头识别记录的数据结构。

用户上传图片获得识别结果过程的时序图如图 4-5 所示。

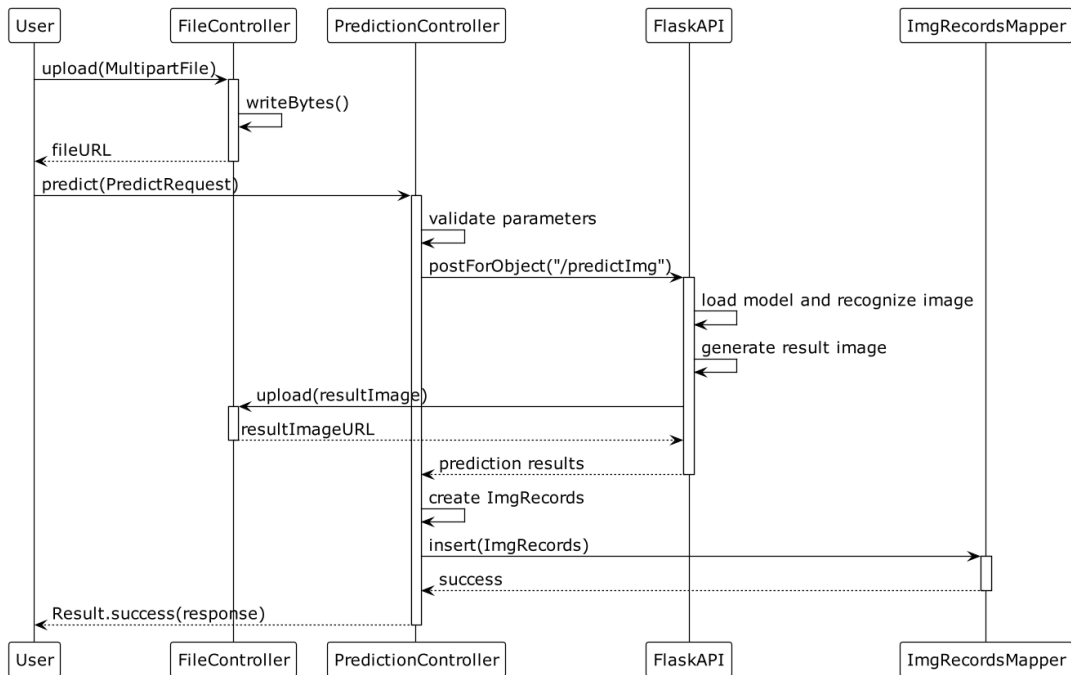


图 4-5 图片识别模块时序图

如图,用户(前端)发起图片上传请求,调用 **FileController** 的 **upload()**方法,把图片保存在 **Java** 服务端,返回图片的 **url** 给前端;前端点击"预测",将图片 **url** 发送给 **PredictionController** 的 **predict()**方法,然后 **PredictionController** 把图片 **url** 发给 **flask** 端的 **predictImg()**方法, **flask** 端调用模型预测,并把预测好的结果图片保存在 **flask** 端,然后把结果图片上传到 **Java** 端,获取结果图的 **url**,返回给 **PredictionController**,最后 **PredictionController** 通过 **Mapper** 将结果 **url** 存到 **MySQL** 的 **imgrecords** 表。对视频的识别、摄像头的识别与此类似。

4.3.2 识别记录管理模块设计

该模块主要负责管理识别记录,包括图片记录、视频记录和摄像头识别记录的查看、修改、下载。对于普通用户,在记录模块可以按病害分类名检索历史记录和选择删除记录。对于管理员,在图片识别记录模块可以对记录进行审核标记,被标记为“忽略”的记录将不再显示,被标记为“已保留”的记录,稍后可以批量导出。导出功能可以对全部“已保留”的图片导出、对最近三天的保留记录导出或者对新保留的记录导出。导出的图片变为“已导出”状态。涉及对 **camerarecords** 表、**imgrecords** 表和 **videorecords** 表的数据管理。

查看图片检索结果模块的静态类图如图 4-6 所示。

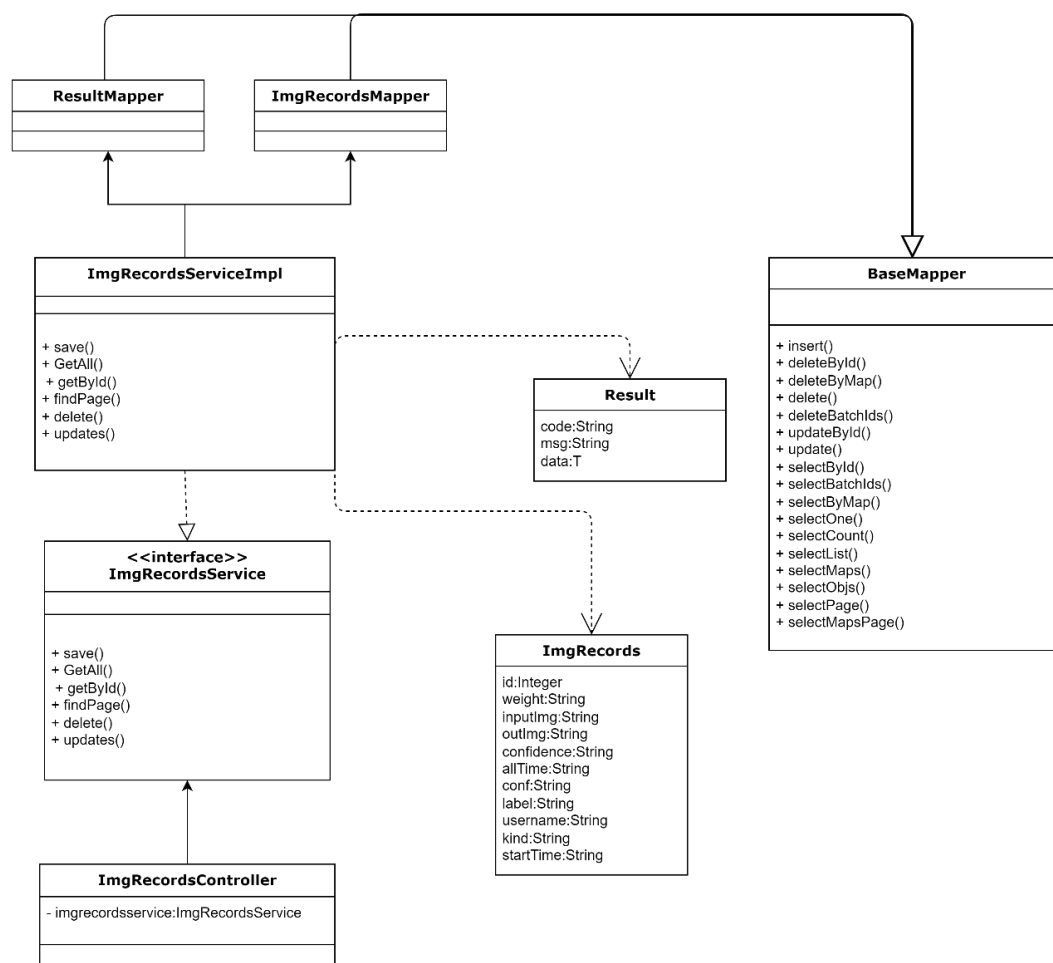


图 4-6 查看检索结果模块类图

本模块主要的核心逻辑在 `ImgRecordsController` 类。`ImgRecordsServiceImpl` 类实现了 `ImgRecordsService` 接口，主要提供了对识别记录的分角色查看、删除和更新功能。在查询与删改的过程中使用了继承自 MyBatis-plus `BaseMapper` 类的 `ImgRecordsMapper` 与 `ResultMapper`，用于在持久化层查询与更新数据。对视频记录、摄像头记录的查询与此类似。

如错误!未找到引用源。所示，管理员对图片记录的管理操作流程如下：

1) 标记优质图片记录

点击“忽略”按钮调用 `onRowIgnore()` 方法，该方法请求 `/ignore` 接口，由 `ImgRecordsController` 的 `ignoreRecord()` 方法处理。点击“保留”按钮调用 `onRowPreserve()` 方法，该方法请求 `/preserve` 接口，由 `ImgRecordsController` 的 `preserveRecord()` 方法处理。

2) 批量导出优质图片记录

通过下拉菜单选择导出类型，触发 `handleExportCommand()` 方法，进而调用 `exportPreservedImages()`。该方法向 `/export-preserved` 接口发起请求，并传递 `onlyNew` 或 `fromDate` 参数。随后根据参数查询标记为 `preserved = true` 的记录，将图片按标签分类

打包成 ZIP，返回供前端下载。

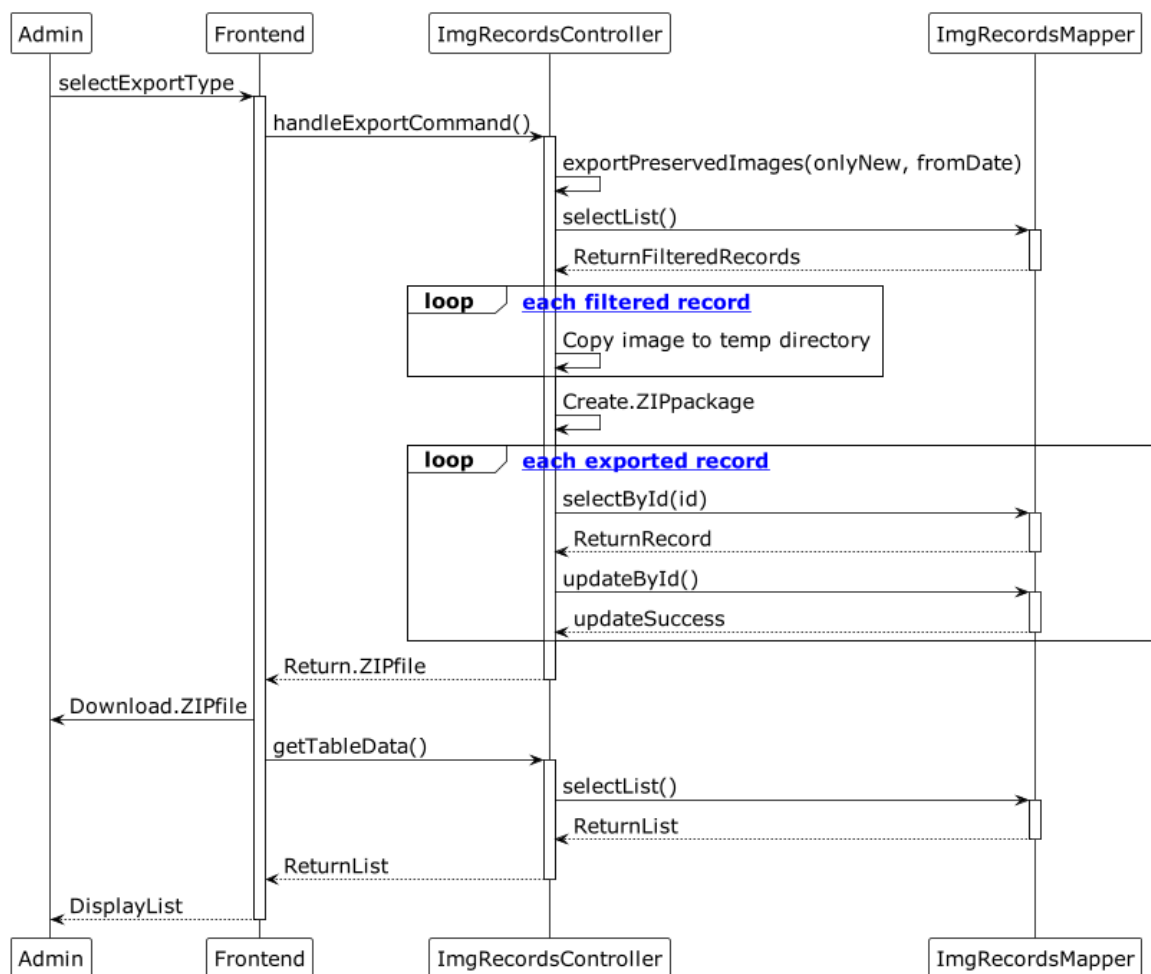


图 4-7 管理员筛选并批量导出记录时序图

4.3.3 用户信息管理模块设计

如图 4-8 所示为该模块类图。

在数据访问层面，`UserMapper` 负责处理与用户表相关的数据库操作，继承了 `MyBatis-Plus` 的 `BaseMapper` 接口，提供了完整的 `CRUD` 操作功能，包括插入、更新、删除、查询等基础数据库操作。该 `Mapper` 还扩展了自定义查询方法，如根据用户名查询用户信息、根据用户名获取密码等特定业务需求的数据访问方法。

在业务逻辑层面，`UserService` 定义了用户管理相关的服务接口，声明了分页查询用户列表、根据用户名查询用户、获取所有用户、更新用户信息、删除用户、保存用户等核心业务方法。而 `ServiceImpl` 则实现了这个接口，提供了用户管理相关的业务逻辑处理，包括用户信息分页查询、用户数据的增删改查等具体实现，并在业务层面处理了查询条件构建、数据验证等逻辑。

另外，`LoginService` 定义了登录认证相关的服务接口，声明了用户登录验证、用户注册、检查用户是否存在等认证业务方法。`LoginServiceImpl` 实现了 `LoginService` 接口，提供了用户登录验证、自动注册、密码校验等业务逻辑的实现。该实现类通过调用

UserMapper 进行数据访问，并封装了登录结果处理逻辑，支持用户不存在时的自动注册流程。

在控制器层面，UserController 作为用户管理的核心控制器，负责处理用户信息管理相关的 HTTP 请求，包括用户列表查询、用户信息更新、用户删除等操作，通过调用 UserService 来实现具体的业务逻辑。LoginController 则专门负责用户认证相关的业务处理，提供了用户登录、用户注册、用户名检查、登录或自动注册等功能接口，通过调用 LoginService 来处理认证业务逻辑。

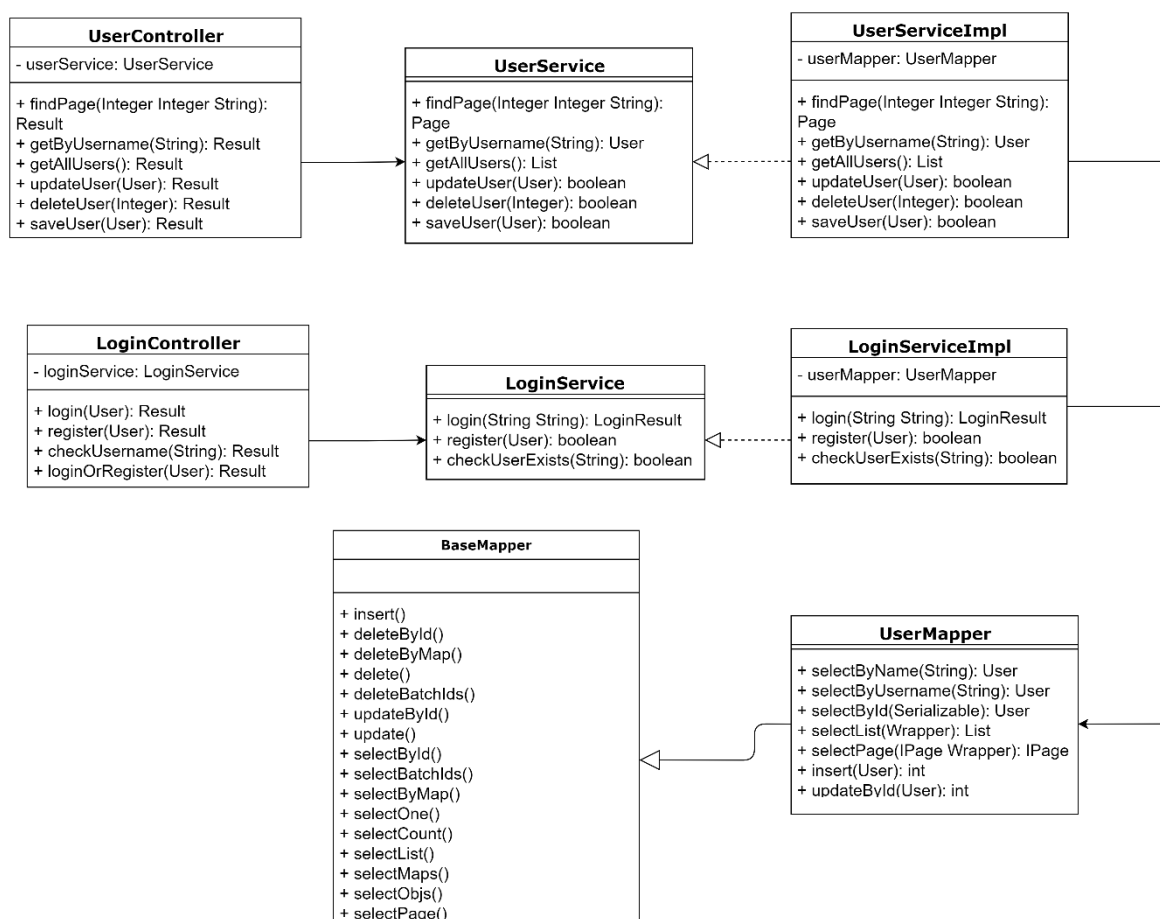


图 4-8 用户信息管理模块类图

用户使用系统前需要完成账号注册。该功能模块主要包含三个部分：登录注册、个人中心 and 用户管理。在注册环节，系统会进行以下验证：首先检查用户名是否已被占用（用户名作为登录凭证必须在系统中保持唯一性）；其次对密码进行加密处理；最后验证必填字段是否完整。

用户登录时，系统会执行身份验证和权限检查。首先，系统会确认用户名是否存在且账户状态正常（未被禁用）。若用户名有效，则进入密码验证环节——由于密码采用加密存储，系统需先将用户输入的密码加密，再与数据库中的密文进行比对。验证通过后，系统会检查验证码的有效性。全部验证通过后，系统将返回用户权限信息，并根据权限级别（普通用户或管理员）开放相应功能。用户登录的时序图如图 4-9 所示。

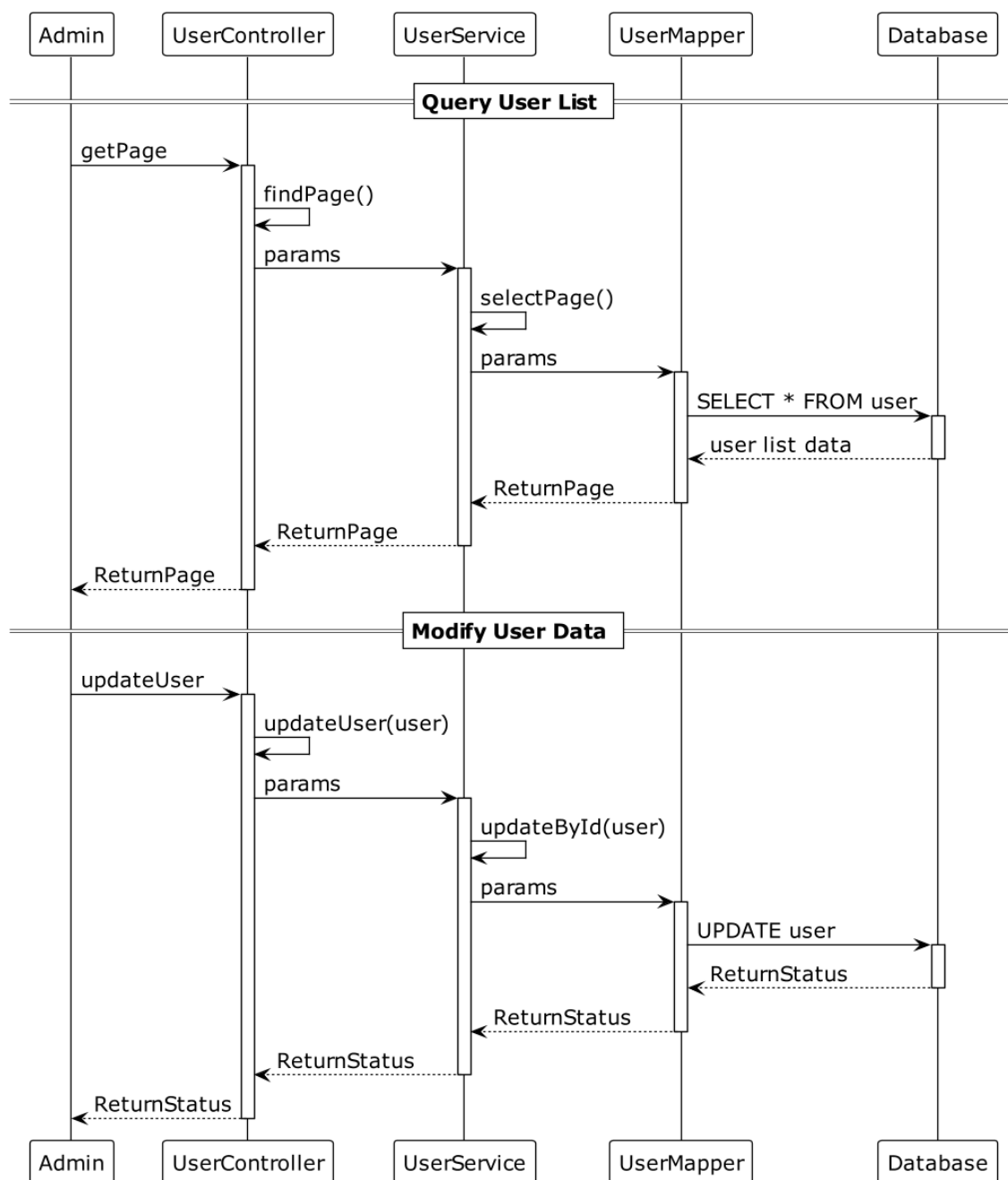


图 4-9 用户登录模块时序图

普通用户与超级管理员均具备账户信息管理能力，主要差异体现在权限范围：个人账户信息对该普通用户可见，所有账户信息仅对管理员可见。

4.4 数据库设计

系统将检测模型处理的数据自动存储至历史档案中。普通用户可对个人检测记录进行查看与删除，管理员则需周期性检查未处理的样本数据，筛选高质量记录并根据需求进行批量下载。系统的核心实体与实体内部的联系如图 4-10 所示。

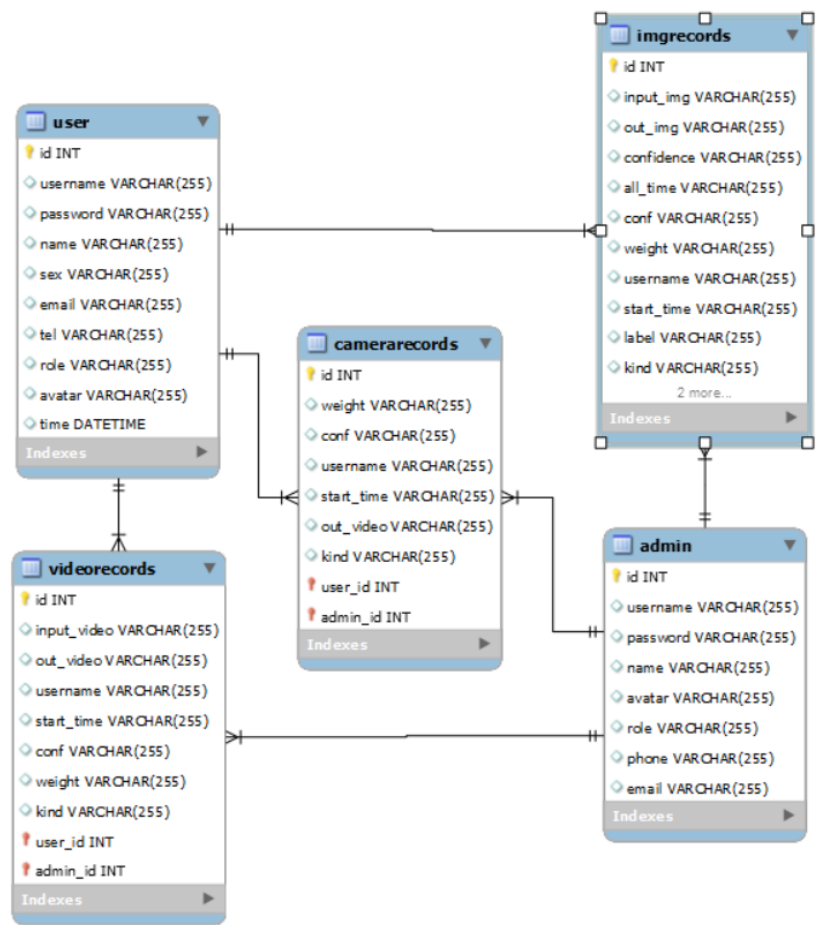


图 4-10 番茄叶片病虫害识别系统数据库 ER 图

1) 管理员账户信息表

Admin 表负责存储管理员账号的详细信息，如表 4-1 所示。

表 4-1 管理员 Admin 表

字段名	字段类型	字段描述	字段约束
id	int	管理员唯一编号	非空、自增、主键
username	varchar(255)	管理员登录用户名	
password	varchar(255)	登录密码	
name	varchar(255)	管理员姓名	
avatar	varchar(255)	头像图片路径	
role	varchar(255)	管理员角色类型	
phone	varchar(255)	联系电话	
email	varchar(255)	电子邮箱地址	

2) 摄像识别记录表

Camerarecords 表负责所有摄像识别记录的详细信息，如表 4-2 所示。

表 4-2 检索结果 Query 表

字段名	字段类型	字段描述	字段约束
-----	------	------	------

id	int	记录唯一编号	非空、自增、主键
weight	varchar(255)	权重文件名称	
conf	varchar(255)	置信度阈值	
username	varchar(255)	操作用户名	
start_time	varchar(255)	操作开始时间	
out_video	varchar(255)	输出视频路径	
kind	varchar(255)	检测种类	

3) 图片识别记录表

Imgrecords 表负责存储所有图片记录的详细信息，如表 4-3 所示。

表 4-3 会话信息 Session 表

字段名	字段类型	字段描述	字段约束
id	int	记录唯一编号	非空、自增、主键
input_img	varchar(255)	输入图像路径	
out_img	varchar(255)	输出图像路径	
confidence	varchar(255)	检测置信度	
all_time	varchar(255)	完整处理耗时	
conf	varchar(255)	置信度阈值	
weight	varchar(255)	使用的模型权重文件	
username	varchar(255)	操作用户名	
start_time	varchar(255)	开始处理时间	
label	varchar(255)	检测目标标签	
kind	varchar(255)	检测类型	
ignored	Boolean	记录是否被忽略	
preserved	Boolean	记录是否被保留	
exported	Boolean	记录是否被导出	
exported_time	varchar(255)	记录导出时间	

4) 用户信息表

User 表负责储存用户的账户信息，如表 4-4 所示。

表 4-4 会话与查询信息 Result 表

字段名	字段类型	字段描述	字段约束
id	int	用户唯一编号	非空、自增、主键
username	varchar(255)	用户名	
password	varchar(255)	登录密码	
name	varchar(255)	用户姓名	
sex	varchar(255)	性别	
email	varchar(255)	电子邮件地址	
tel	varchar(255)	联系电话	
role	varchar(255)	用户角色	

avatar	varchar(255)	头像图片路径
time	datetime	注册时间

5) 视频识别记录表

Videorecords 表负责储存所有图片记录的详细信息，如表 4-5 所示。

表 4-5 音频信息 Audio 表

字段名	字段类型	字段描述	字段约束
id	int	视频记录唯一编号	非空、自增、主键
input_video	varchar(255)	输入视频文件路径	
out_video	varchar(255)	输出视频文件路径	
username	varchar(255)	操作用户名	
start_time	varchar(255)	开始处理时间	
conf	varchar(255)	置信度阈值	
weight	varchar(255)	模型权重文件	
kind	varchar(255)	检测类型	

4.5 本章小结

本章针对基于深度学习技术的番茄叶片病虫害检测系统开展了功能需求分析工作，重点从并发处理性能、识别精度指标以及系统响应时间等三个维度深入探讨了非功能性需求特征。依据上述需求分析的成果，本章制定了系统的总体设计方案，按照功能特性将整个系统架构划分为三个核心模块：病虫害识别检索模块、识别结果记录管理模块以及用户账户管理模块。在具体设计环节中，采用 UML 类图和 UML 时序图等建模工具对前述各项功能进行了详细的设计描述与功能扩展，并在最终阶段完成了系统数据库表结构的设计工作。

5 番茄叶片病虫害识别系统的实现与测试

5.1 系统开发环境

本系统采用了 B/S 架构，前后端分离开发方式。算法实现语言采用了 Python3.8，Web 后端使用了 Springboot 框架架构配合 Flask 框架调用 YOLOv8 模型，使用 Mysql 数据库持久化数据；前端网页主要采用了 vue300 +ElementUI 技术。系统的开发环境配置如表 5-1 所示。

表 5-1 系统开发环境配置

组件	参数
服务器端 CPU	Intel(R) Core(TM) i5-1035H CPU @ 1.00GHz
服务器端内存	16GB
服务器端 GPU	NVIDIA GeForce MX 350
服务器端硬盘	250GB
服务器操作系统	Windows 10 64 位
数据库及版本	Mysql 8.0.11
前端语言及框架	Vue3.0 / ElementUI
后端语言及框架	Java17.0.5 / SpringBoot3.0.5 / Mybatis-plus 3.5.3
开发软件版本	IntelliJ IDEA 2022.3.2, Visual Studio Code 1.99.1
客户端浏览器配置	chrome 125.0.6422.60 及以上

5.2 系统功能模块实现

5.2.1 识别搜索模块实现

用户(前端)发起图片上传请求，调用 FileController 的 upload()方法,把图片保存在 Java 服务端，返回图片的 url 给前端；前端设置置信度，点击"预测"，将图片 url 发送给 PredictionController 的 predict()方法，然后 PredictionController 把图片 url 发给 flask 端的 predictImg()方法，flask 端调用模型预测，并把预测好的结果图片保存在 flask 端，然后把结果图片上传到 Java 端，获取结果图的 url，返回给 PredictionController，最后 PredictionController 把结果 url 存到 MySQL 的 imgrecords 表。对视频的识别、摄像头的识别与此类似。图 5-1、图 5-2 展示了上传文件、点击识别显示识别结果的过程。

Java 端的 PredictionController 核心代码如下：

```
public Result<?> predict(@RequestBody PredictRequest request) {
    if (request == null || request.getInputImg() == null || request.getInputImg().isEmpty()) {
        return Result.error("-1", "未提供图片链接");
    } else if (request.getWeight() == null || request.getWeight().isEmpty()) {
        return Result.error("-1", "未提供权重");
    }
}
```

```

try {
    // 创建请求体
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    HttpEntity<PredictRequest> requestEntity = new HttpEntity<>(request, headers);

    // 调用 Flask API
    String response = restTemplate.postForObject("http://localhost:5000/predictImg",
requestEntity, String.class);
    System.out.println("java 发的预测请求收到 flask 响应: Received response: \n" + response);
    JSONObject responses = JSONObject.parseObject(response);
    if(responses.get("status").equals(400)){
        return Result.error("-1", "Error: " + responses.get("message"));
    }else {
//        把图片识别记录保存
        ImgRecords imgRecords = new ImgRecords();
        imgRecords.setWeight(request.getWeight());
        imgRecords.setConfidence(String.valueOf(responses.get("confidence")));
        imgRecords.setAllTime(String.valueOf(responses.get("allTime")));
        imgRecords.setOutImg(String.valueOf(responses.get("outImg")));
        imgRecordsMapper.insert(imgRecords); // 插入到数据库
        return Result.success(response);
    }
}

```

Flask 端负责接受 Java 端发出的预测请求，将其中包含的待识别图片 url 输入到加载的模型，其中 predictImg.py 脚本负责处理图片预测请求，main.py 脚本负责处理视频与摄像预测请求。识别完成后，Flask 端保存识别结果并上传到 Java 服务器，最后把结果图片 url 返回。predictImg.py 核心代码如下：

```

class ImagePredictor:
    def __init__(self, weights_path, img_path, kind, save_path="./runs/result.jpg", conf=0.5):
        """
        初始化 ImagePredictor 类
        :param weights_path: 权重文件路径
        :param img_path: 输入图像路径
        :param save_path: 结果保存路径
        :param conf: 置信度阈值
        """
        self.model = YOLO(weights_path)
        self.conf = conf
        self.img_path = img_path
        self.save_path = save_path
        self.kind = {
            'rice': ['Brown_Spot（褐斑病）', 'Rice_Blast（稻瘟病）', 'Bacterial_Blight（细菌性叶

```



```

枯病）'],
        'corn': ['blight（疫病）', 'common_rust（普通锈病）', 'gray_spot（灰斑病）', 'health
（健康）'],
        'strawberry': ['Angular Leafspot（角斑病）', 'Anthracnose Fruit Rot（炭疽果腐病）',
'Blossom Blight（花枯病）', 'Gray Mold（灰霉病）', 'Leaf Spot（叶斑病）', 'Powdery Mildew Fruit（白
粉病果）', 'Powdery Mildew Leaf（白粉病叶）'],
        'tomato': ['Early Blight（早疫病）', 'Healthy（健康）', 'Late Blight（晚疫病）', 'Leaf
Miner（潜叶病）', 'LeafMold（叶霉病）', 'Mosaic Virus（花叶病毒）', 'Septoria（壳针孢属）', 'Spider
Mites（蜘蛛螨）', 'Yellow Leaf Curl Virus（黄化卷叶病毒）']
    }
    self.labels = self.kind[kind]

def predict(self):
    """
    预测图像并保存结果
    """
    start_time = time.time() # 开始计时

    # 执行预测
    results = self.model(source=self.img_path, conf=self.conf, half=True, save_conf=True)

    end_time = time.time() # 结束计时
    elapsed_time = end_time - start_time # 计算用时

    all_results = {
        'labels': [], # 存储所有标签
        'confidences': [], # 存储所有置信度
        'allTime': f'{elapsed_time:.3f}秒'
    }
    return all_results
    # 将每个结果保存到字典中
    for label, conf in predictions:
        all_results['labels'].append(label)
        all_results['confidences'].append(f'{conf * 100:.2f}%')

    result.save(filename=self.save_path) # 保存结果
}
return all_results

```

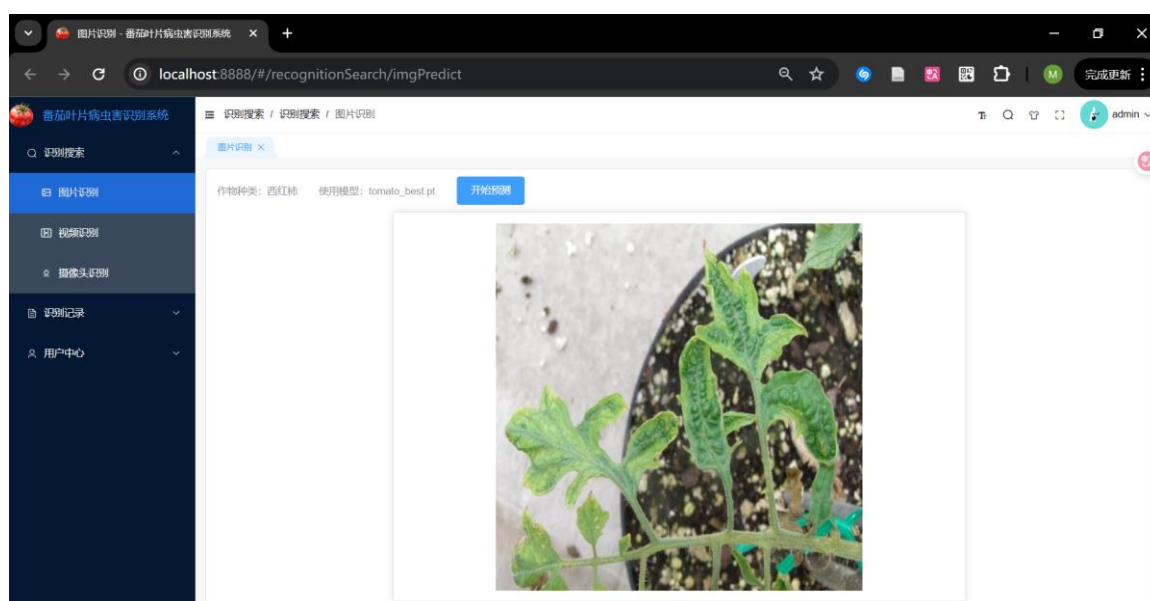


图 5-1 上传待识别图片

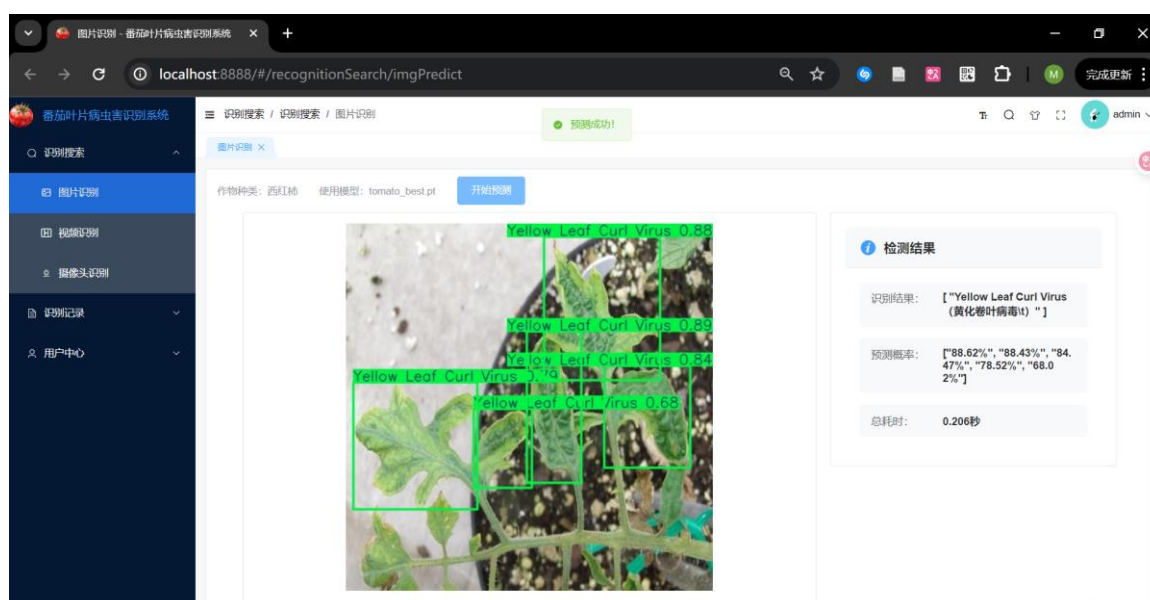


图 5-2 “图片检测” 点击“开始检测”结果显示

对于开启摄像头读取视频流进行识别，用户首先点击“摄像检测”，打开摄像功能选项卡，再点击“开始录制”，此时摄像头画面中会实时显示病害种类的标签框，如图 5-3 所示。

视频流的识别主要是由前端页面直接请求 Flask 的 5000 端口，开启摄像头后将图像帧输入到算法模型中，将模型处理好的带有病虫害类别标签框的结果帧返回给前端显示，当用户想要结束摄像识别，点击“结束录制”将会调用 `stopCamera()` 终止识别。Flask 端 `main.py` 中的 `predictCamera()`, `stopCamera()` 方法代码如下：

```
def predictCamera(self):
    """摄像头视频流处理接口"""
    self.data.clear()
```

```

self.data.update({
    "username": request.args.get('username'), "weight": request.args.get('weight'),
    "kind": request.args.get('kind'),
    "conf": request.args.get('conf'), "startTime": request.args.get('startTime')
})
self.socketio.emit('message', {'data': '正在加载，请稍等！'})
model = YOLO(f'./weights/{self.data["weight"]}')
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
video_writer = cv2.VideoWriter(self.paths['camera_output'], cv2.VideoWriter_fourcc(*'XVID'),
20, (640, 480))

self.socketio.emit('message', {'data': '处理完成，正在保存！'})
for progress in self.convert_avi_to_mp4(self.paths['camera_output']):
    self.socketio.emit('progress', {'data': progress})
uploadedUrl = self.upload(self.paths['output'])
self.data["outVideo"] = uploadedUrl
print(self.data)
self.save_data(json.dumps(self.data), 'http://localhost:9999/cameraRecords')
self.cleanup_files([self.paths['download'], self.paths['output'],
self.paths['camera_output']])

```

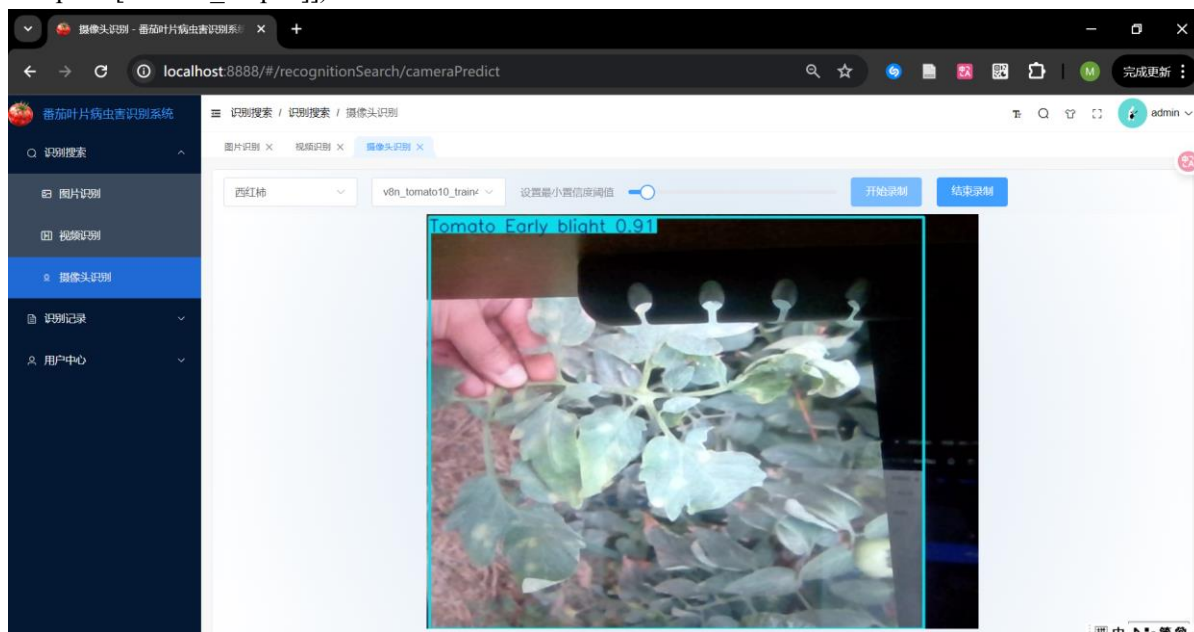


图 5-3 摄像头识别结果显示

5.2.2 识别记录管理模块实现

用户通过上传图片、视频或开启摄像头识别病虫害后，点击图片识别记录、视频识别记录和摄像识别记录查看历史记录，如图 5-4 所示。对于超级管理员，则可以对所有图片记录进行保留标记、忽略标记，并按条件批量导出保留的记录。

记录的展示由 Java 端通过 Mapper 与 Records 数据表交互进行查询、返回查询结果并在前端渲染完成。以图片记录展示为例，Java 代码如下：

```

@PostMapping("/predict")
public Result<?> predict(@RequestBody PredictRequest request) {
    try {
        // 调用 Flask API
        String response = restTemplate.postForObject("http://localhost:5000/predictImg",
requestEntity, String.class);
        System.out.println("java 发的预测请求收到 flask 响应: Received response: \n" + response);
        JSONObject responses = JSONObject.parseObject(response);
        if(responses.get("status").equals(400)){
            return Result.error("-1", "Error: " + responses.get("message"));
        }else {
            // 把图片识别记录保存
            ImgRecords imgRecords = new ImgRecords();
            imgRecords.setWeight(request.getWeight());
            imgRecords.setConf(request.getConf());
            imgRecordsMapper.insert(imgRecords); // 插入到数据库
            return Result.success(response);
        }
    } catch (Exception e) {
        return Result.error("-1", "Error: " + e.getMessage());
    }
}

```

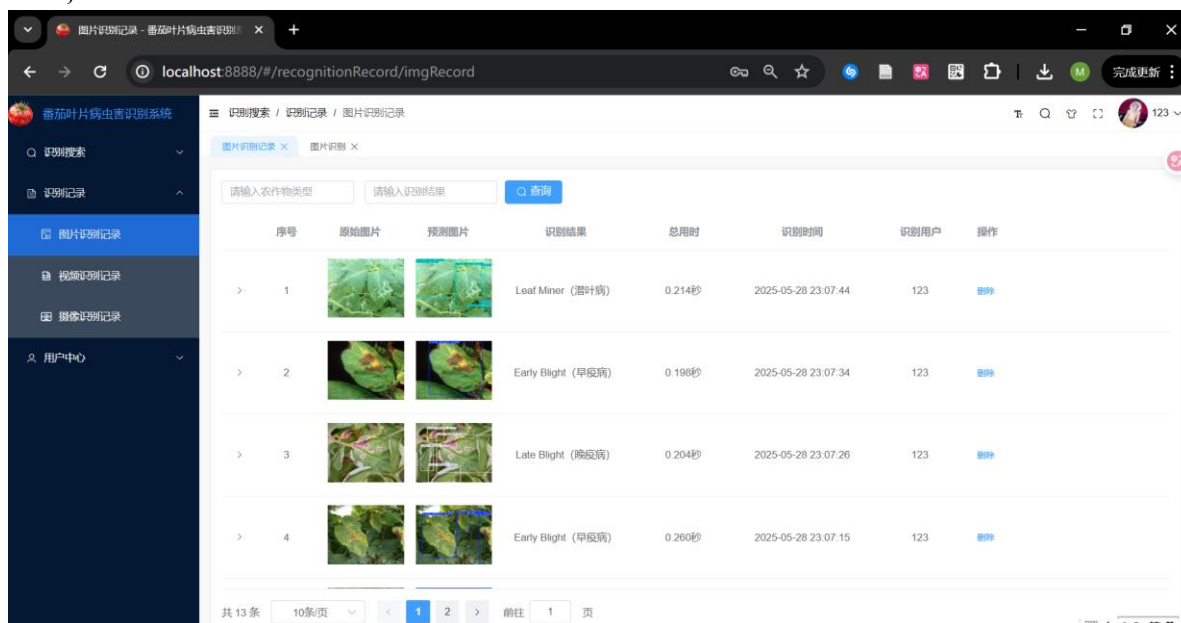


图 5-4 普通用户的图片识别记录查看

管理员对图片记录的管理，包括查看、标记、导出。具体而言包括按作物类型、识别结果等条件筛选查看图片记录，控制是否显示已忽略的记录，同时具备对记录进行“忽略”或“保留”标记的权限，可选择批量导出全部、仅新增或近期的已保留图片。管理员标记图片、批量导出结果如图 5-5 至 5-7 所示。

后端控制器负责接收并处理这些请求：根据前端传递的筛选条件和用户角色（管理员/普通用户）查询返回相应记录，处理记录状态的更新，以及执行批量导出任务—

—查询符合条件的已保留记录，将图片按标签分类打包成 ZIP 文件，更新导出状态和时间戳，最后将文件流返回给前端供用户下载。管理员标记、批量导出的代码如下：

```

@PutMapping("/{id}/ignore")
public Result<?> ignoreRecord(@PathVariable Integer id) {
    ImgRecords record = imgRecordsMapper.selectById(id);
    if (record != null) {
        record.setIgnored(true);
        imgRecordsMapper.updateById(record);
        return Result.success();
    }
    return Result.error("-1", "记录不存在");
}

@DeleteMapping("/{id}")
public Result<?> delete(@PathVariable int id) {
    imgRecordsMapper.deleteById(id);
    return Result.success();
}

@PostMapping("/update")
public Result<?> updates(@RequestBody ImgRecords imgrecords) {
    imgRecordsMapper.updateById(imgrecords);
    return Result.success();
}

@PutMapping("/{id}/preserve")
public Result<?> preserveRecord(@PathVariable Integer id) {
    ImgRecords record = imgRecordsMapper.selectById(id);
    if (record != null) {
        record.setPreserved(true);
        imgRecordsMapper.updateById(record);
        return Result.success();
    }
    return Result.error("-1", "记录不存在");
}

@PostMapping
public Result<?> save(@RequestBody ImgRecords imgrecords) {
    imgrecords.setIgnored(false); // 新纪录默认未忽略
    imgrecords.setPreserved(false); // 新记录默认未保留
    System.out.println(imgrecords);
    imgRecordsMapper.insert(imgrecords);
    return Result.success();
}

@GetMapping("/export-preserved")

```

```

public void exportPreservedImages(HttpServletRequest request, HttpServletResponse response,
                                   @RequestParam(defaultValue = "false") Boolean onlyNew,
                                   @RequestParam(defaultValue = "") String fromDate) {
    try {
        // 查询已保留的记录
        LambdaQueryWrapper<ImgRecords> wrapper = Wrappers.<ImgRecords>lambdaQuery()
            .eq(ImgRecords::getPreserved, true);

        // 筛选条件：仅未导出的或者从指定日期以来的
        if (onlyNew) {
            wrapper.eq(ImgRecords::getExported, false).or().isNull(ImgRecords::getExported);
        }

        // 如果指定了日期，则只导出该日期之后的记录
        if (StrUtil.isNotBlank(fromDate)) {
            wrapper.ge(ImgRecords::getStartTime, fromDate);
        }

        List<ImgRecords> preservedRecords = imgRecordsMapper.selectList(wrapper);

        if (preservedRecords.isEmpty()) {
            response.setContentType("application/json");
            response.setCharacterEncoding("UTF-8");
            response.getWriter().write("{\"code\":-1,\"msg\":\"没有需要导出的记录\"}");
            return;
        }

        ObjectMapper objectMapper = new ObjectMapper();
        Map<String, Integer> exportCounts = new HashMap<>();

        // 记录此次导出的记录 ID
        List<Integer> exportedIds = new ArrayList<>();

        // 创建一个临时导出目录
        String timestamp = String.valueOf(System.currentTimeMillis());
        Path tempExportDir = Paths.get(System.getProperty("java.io.tmpdir"), "preserved_export_"
+ timestamp);
        Files.createDirectories(tempExportDir);

        for (ImgRecords record : preservedRecords) {
            String inputImgPath = record.getInputImg();
            if (inputImgPath == null || inputImgPath.isEmpty()) {
                continue;
            }

            String[] labels = objectMapper.readValue(record.getLabel(), String[].class);

```

```

        if (labels == null || labels.length == 0) {
            continue;
        }
        // 清理标签名称
        String firstLabel = sanitizeFileName(labels[0]);

        Path labelDir = tempExportDir.resolve(firstLabel);
        Files.createDirectories(labelDir);

        Path sourcePath = Paths.get(inputImgPath.replace("http://localhost:9999/", ""));
        if (!Files.exists(sourcePath)) {
            continue;
        }

        Integer count = exportCounts.getOrDefault(firstLabel, 0) + 1;
        exportCounts.put(firstLabel, count);

        String extension = inputImgPath.substring(inputImgPath.lastIndexOf('.'));
        String targetFileName = firstLabel + "_" + count + extension;
        Path targetPath = labelDir.resolve(targetFileName);

        Files.copy(sourcePath, targetPath, StandardCopyOption.REPLACE_EXISTING);

        // 添加到已导出记录列表
        exportedIds.add(record.getId());
    }

    // 设置响应头
    response.setContentType("application/zip");
    String fileName = onlyNew ? "new_preserved_images_" + timestamp + ".zip" :
    "all_preserved_images_" + timestamp + ".zip";
    response.setHeader("Content-Disposition", "attachment; filename=" + fileName);

    // 直接将 ZIP 写入响应流
    try (ZipOutputStream zos = new ZipOutputStream(response.getOutputStream())) {
        Files.walk(tempExportDir).filter(Files::isRegularFile).forEach(path -> {
            try {
                String zipEntryName = tempExportDir.relativeTo(path).toString();
                zos.putNextEntry(new ZipEntry(zipEntryName));
                Files.copy(path, zos);
                zos.closeEntry();
            } catch (IOException e) {
                e.printStackTrace();
            }
        });
    }
}

```

```
// 删除临时目录
deleteDirectory(tempExportDir.toFile());

// 更新导出状态和时间
String currentTime = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new
Date());

for (Integer id : exportedIds) {
    ImgRecords record = imgRecordsMapper.selectById(id);
    record.setExported(true);
    record.setExportTime(currentTime);
    imgRecordsMapper.updateById(record);
}

} catch (Exception e) {
    e.printStackTrace();
    try {
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
        response.getWriter().write("{\"code\":-1,\"msg\"\":\"导出失败：\" + e.getMessage() +
\"\"}");
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
}
```

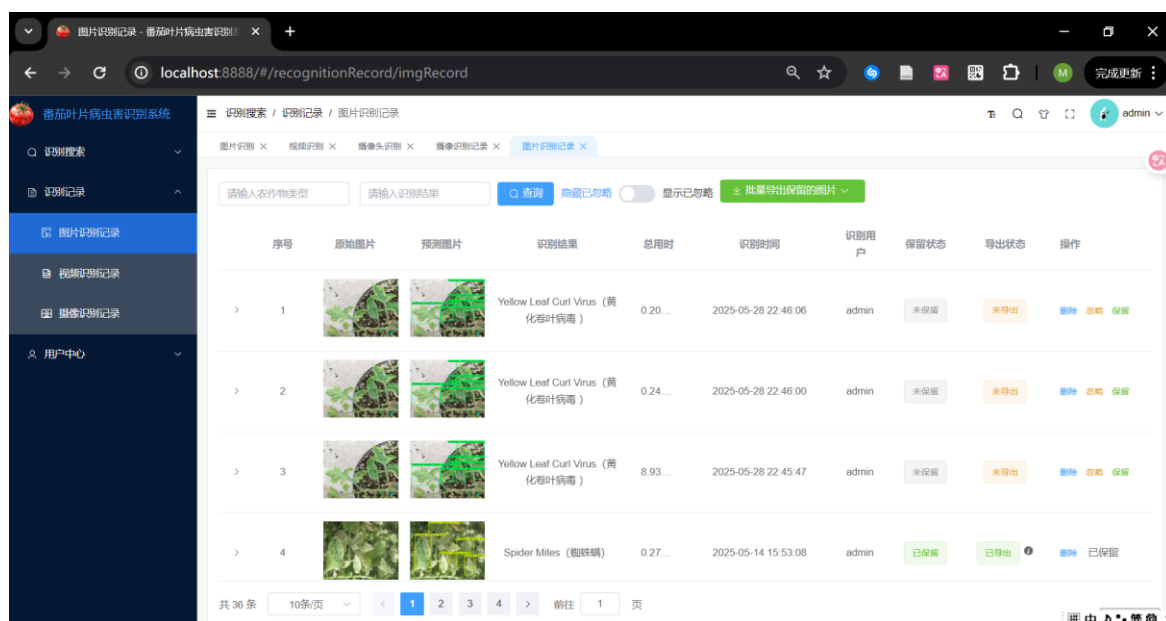


图 5-5 管理员管理图片记录页面

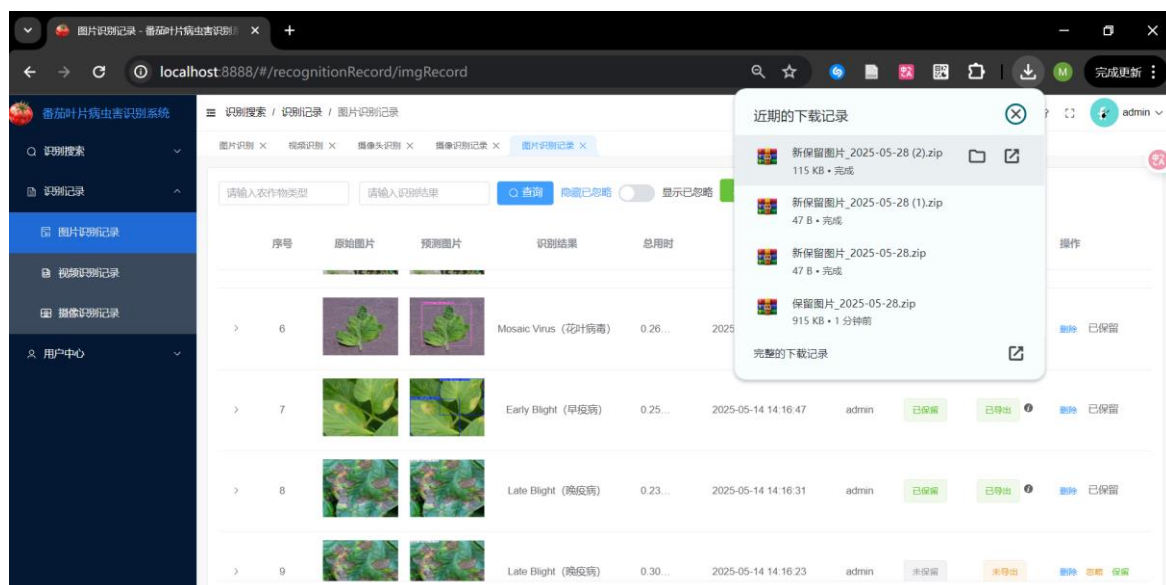


图 5-6 图片记录批量导出

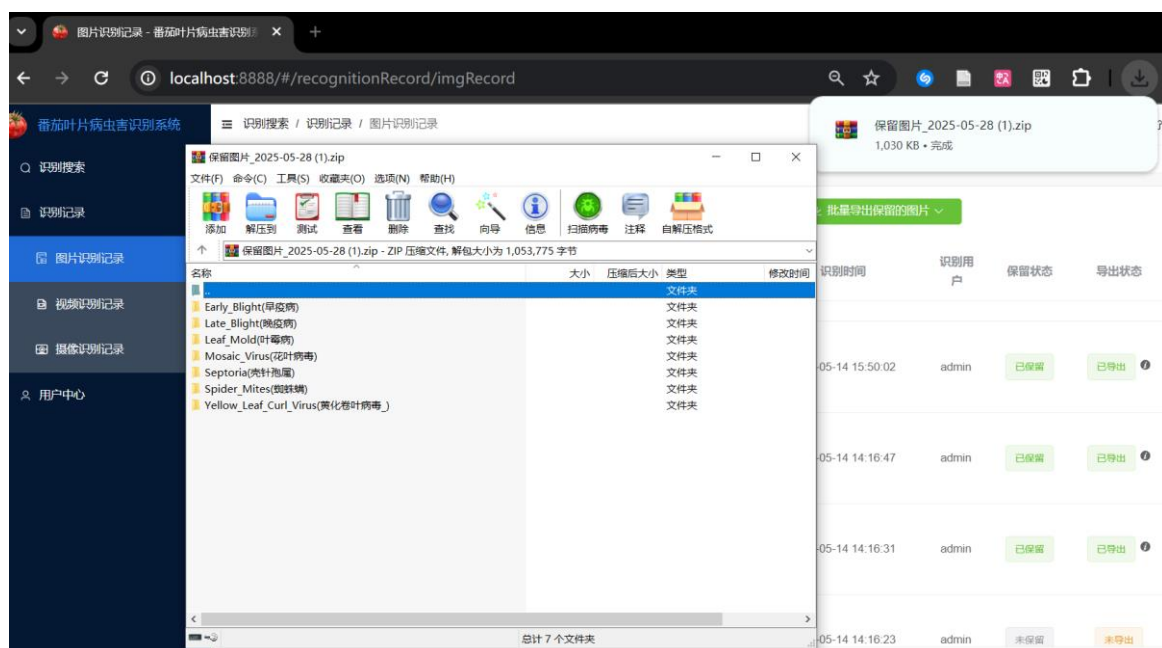


图 5-7 按图像类别批量导出结果

5.2.3 用户信息管理模块实现

本小节展示用户管理模块的主要功能实现。当登录用户角色为管理员时，可以使用用户管理功能。该模块通过 `UserController` 类提供了用户的分页查询、注册、登录、修改、删除等基础接口。其中，`findPage()`方法支持根据用户名关键词分页模糊查询用户数据，并按 ID 倒序排序；`login()`方法验证用户名与密码是否匹配，若不匹配则返回相应错误信息；`register()`方法用于新用户注册，若用户名已存在则返回错误提示，若无冲突则生成默认用户信息并写入数据库。此外，模块还提供了新增（`save()`）、修改（`updates()`）、删除（`delete()`）以及按用户名查询（`getByUsername()`）和获取全部用户

列表（GetAll()）等接口。用户管理界面如图 5-8 所示。管理员对所有用户的查看和增删代码如下：

```
public Result<?> findPage(@RequestParam(defaultValue = "1") Integer pageNum,
    LambdaQueryWrapper<User> wrapper = Wrappers.<User>lambdaQuery();
    wrapper.orderByDesc(User::getId);
    if (StrUtil.isNotBlank(search)) {
        wrapper.like(User::getUsername, search);
    }
    Page<User> UserPage = userMapper.selectPage(new Page<>(pageNum, pageSize), wrapper);
    return Result.success(UserPage);
}

@GetMapping("/{username}")
public Result<?> getByUsername(@PathVariable String username) {
    System.out.println(username);
    return Result.success(userMapper.selectByUsername(username));
}

@PostMapping
public Result<?> save(@RequestBody User user) {
    userMapper.insert(user);
    return Result.success();
}
```

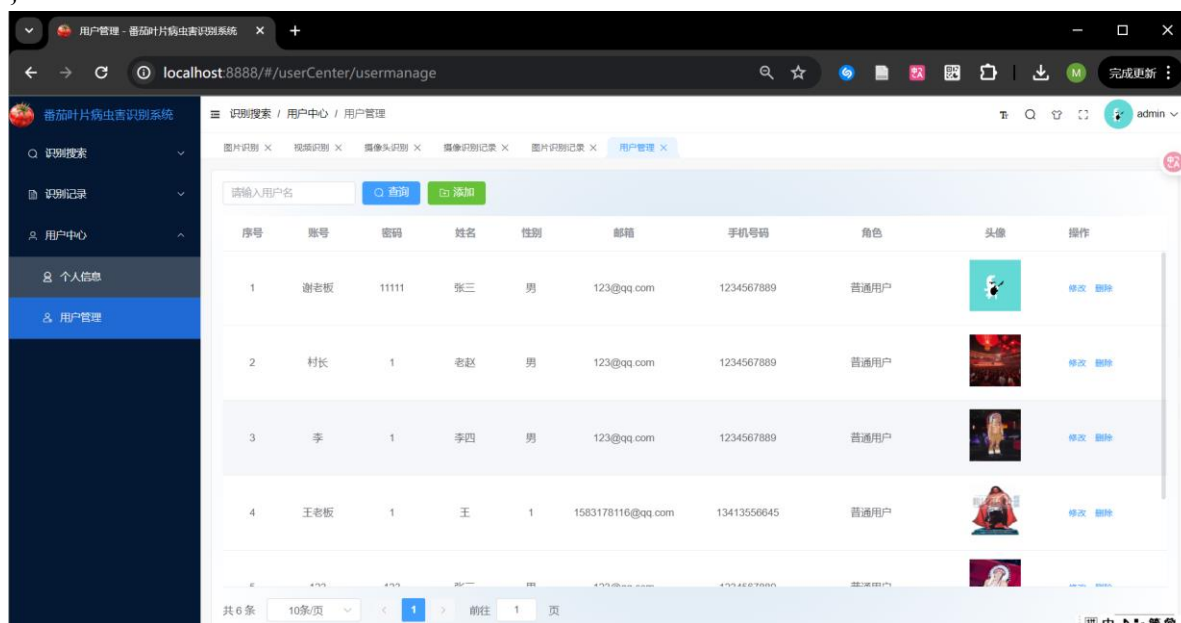


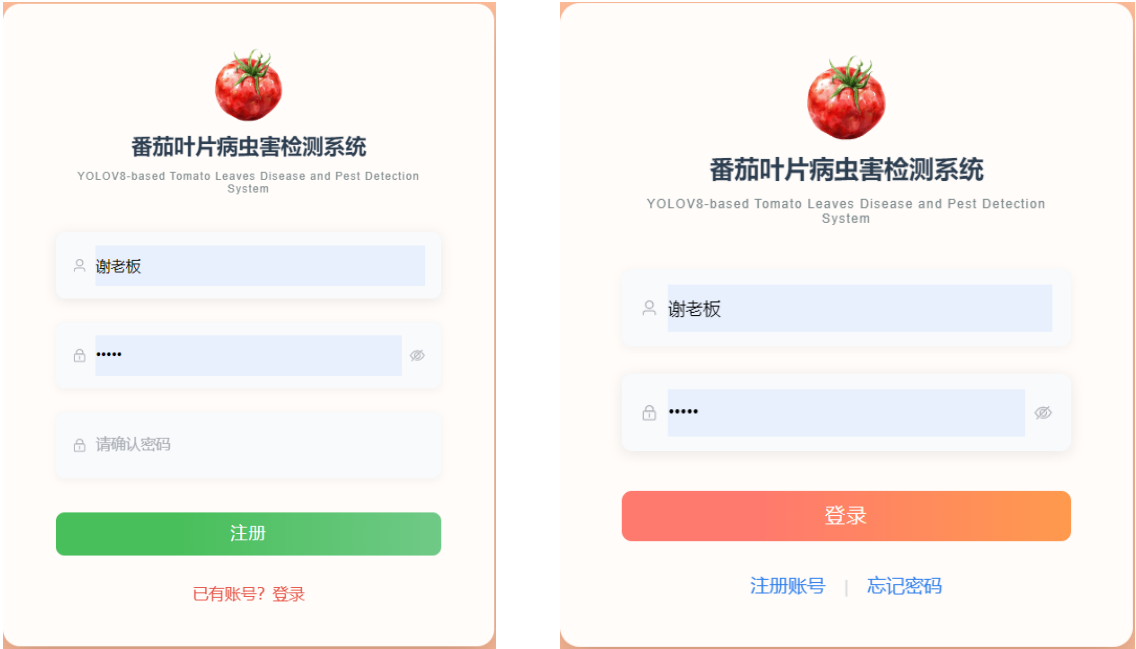
图 5-8 管理员进行用户管理

普通用户注册通过@PostMapping("/register")实现。接口接收用户名、密码和角色信息后进行非空校验，限制仅注册普通用户角色，检查用户名是否已存在，并用正则验证密码强度（不少于 8 位且包含字母数字）。最后通过 BCryptPasswordEncoder 加密密码并存入数据库。

```
@PostMapping("/register")
```

```
public Result register(@RequestBody Account account) {
    // 1. 参数完整性校验
    if (StringUtil.isBlank(account.getUsername()) || StringUtil.isBlank(account.getPassword())
        || ObjectUtil.isEmpty(account.getRole())) {
        return Result.error(ResultCodeEnum.PARAM_LOST_ERROR);
    }
    MimaEncoder encoder = new MimaEncoder();
    account.setPassword(encoder.encode(password));
    studentService.register(account);
    return Result.success();
}
```

用户登录时，前端先验证用户名密码非空性，后端通过 `userService.login()` 方法验证凭据。验证通过后，系统根据用户角色调用 `JwtUtils.generateToken()` 生成包含用户名和角色信息的加密 JWT Token。该 Token 返回给前端用于维持登录状态，并在后续请求头中用于身份验证。登录、注册界面如图 5-9 所示。



(a) 注册界面

(b) 登录界面

图 5-9 用户注册、登录界面

5.3 系统测试

5.3.1 测试环境

针对本文构建的番茄叶片病虫害检测系统，测试验证工作选用了 64 位 Windows10 系统作为运行平台，并通过 Google Chrome 浏览器执行系统调试任务，相关测试环境的具体配置信息详见表 5-2。

表 5-2 番茄叶片病虫害检测系统测试环境配置表

组件	参数
----	----

测试 PC CPU	Intel(R) Core(TM) i5-1035H CPU @ 1.00GHz
测试 PC 内存	16GB
测试 PC 硬盘	250GB
测试 PC 操作系统	Windows 10 64 位
测试 PC 浏览器及版本	chrome 125.0.6422.60 及以上

5.3.2 功能测试

系统测试采用黑盒测试的方式，测试只关注各个子模块的输入、输出与运行时状态，并将预期结果与实际输出对比，来确定功能测试是否完成。主要结果如表 5-3、表 5-4 管理员管理数据功能测试表表 5-4 所示。

表 5-3 用户通用功能测试表

测试用例	操作	预期结果	结论
页面切换测试	点击顶部导航栏或侧边栏导航按钮	页面正常切换，展示对应模块内容	测试通过
用户登录测试	输入用户名与密码后点击登录按钮	登录成功，跳转到系统主页	测试通过
用户注册测试	在注册页面填写信息后点击注册按钮	注册成功，提示跳转到登录页面	测试通过
上传图片测试	在图片识别页面选择图片后点击上传按钮	图片成功上传，后端接收文件	测试通过
上传视频测试	在视频识别页面选择视频文件点击上传	视频上传成功，后端接收并提取帧图	测试通过
图片识别测试	上传图片后点击“开始识别”按钮	系统返回识别结果并展示	测试通过
视频识别测试	上传视频后点击“开始识别”按钮	系统返回识别结果并展示	测试通过
查询个人图片识别记录测试	进入“识别记录”页面选择“图片记录”	系统展示该用户上传过的图片识别记录	测试通过
查询个人视频识别记录测试	进入“识别记录”页面选择“视频记录”	系统展示该用户的视频识别记录	测试通过
查询个人摄像识别记录测试	进入“识别记录”页面选择“摄像记录”	系统展示该用户的摄像头识别记录	测试通过
删除图片识别记录测试	在图片识别记录表中点击删除按钮	对应记录删除，表格实时更新	测试通过
删除视频识别记录测试	在视频识别记录表中点击删除按钮	对应记录删除，表格实时更新	测试通过

删除摄像识别记录测试	在摄像记录表中点击删除按钮	对应记录删除，表格实时更新	测试通过
更改个人账户信息测试	进入“个人信息”页面修改信息后保存	修改成功，提示更新完成	测试通过

表 5-4 管理员管理数据功能测试表

测试用例	操作	预期结果	结论
查询所有用户图片识别记录测试	管理端进入“图片识别记录管理”页面	显示所有用户的图片识别记录	通过
查询所有用户视频识别记录测试	管理端进入“视频识别记录管理”页面	显示所有用户的视频识别记录	通过
查询所有用户摄像识别记录测试	管理端进入“摄像识别记录管理”页面	显示所有用户的摄像识别记录	通过
标记记录为“已保留”	记录管理页面点击“保留”	该记录显示为“已保留”	通过
标记记录为“已忽略”	记录管理页面点击“忽略”	该记录显示为“已忽略”	通过
将保留的记录批量导出	记录管理页面点击“批量导出”	浏览器弹出下载 zip 文件	通过
添加用户测试	在用户管理页点击“新增用户”，填写表单并提交	用户添加成功，表格更新	通过
删除用户测试	在用户管理表格中点击删除图标	用户删除成功，表格实时更新	通过
修改用户测试	在用户管理页点击编辑按钮修改信息	用户信息更新成功，提示修改完成	通过

通过上述测试可知，所有测试用例均通过了测试。系统实现了需求分析中的功能需求，且各项功能均通过功能性测试。

5.3.3 非功能性测试

由系统的非功能性需求分析，对系统的非功能性测试主要是对响应时间、识别准确率进行测试。

1) 系统响应时间测试

测试场景主要涵盖以下核心功能模块：待检测图像文件上传处理、病害识别历史记录查询浏览、以及用户权限管理操作等关键业务流程。测试环境模拟真实使用场景，设定 50 名并发用户同时在客户端执行不同类型的操作请求。

针对系统各个功能页面的响应时间进行多轮次测试，每个测试项目重复执行多次以确保数据的可靠性，并计算各页面响应时间的算术平均值作为最终评估指标。通过系统性的性能测试，获得了各功能模块的平均页面响应时间数据，具体测试结果如表 5-5 所示。

表 5-5 系统响应时间测试表

测试项	响应时间	预期响应时间	结论
切换至首页	< 0.3s	< 0.5s	测试通过

切换至识别记录页面	< 0.3s	< 0.5s	测试通过
上传图片文件	< 0.8s	< 1s	测试通过
上传视频文件	< 1.2s	< 1.5s	测试通过
查询个人识别记录	< 0.5s	< 1s	测试通过
管理员查询所有用户记录	< 0.8s	< 1s	测试通过
登录系统	< 0.3s	< 0.8s	测试通过
注册账号	< 0.3s	< 0.8s	测试通过

测试数据表明，系统在各项性能指标上均达到了预期的设计要求。具体而言，针对页面导航切换类操作，包括主界面跳转、病害识别功能模块访问、历史记录管理界面以及用户权限管理页面等基础交互操作，系统平均响应延迟控制在 0.3 秒以内，显著优于设定的 0.5 秒性能基准。

在数据库交互密集型操作方面，涵盖个人识别历史查询、全局用户记录检索、数据恢复处理、记录删除以及检测结果文件下载等功能，系统响应时间保持在 0.8 秒以下，满足了 1 秒响应时限的性能约束条件。

2) 模型识别精度测试

本测试中使用的测试图片与模型实验中的数据相同，在一次识别过程中，模型所输出的所有检测结果中，实际为目标的数量称为真正例数量（True Positives）；而全部被识别为目标的数量称为预测为正的总数。将真正例数量与预测为正的总数之比，称为一次识别的准确率（Precision）。表 5-6 列出了每一类病虫害的准确率。

表 5-6 模型识别精度测试结果

类别	图像数	准确率（P）
全部	3633	0.940
番茄细菌斑点病	376	0.928
番茄早疫病	221	0.967
番茄晚疫病	374	0.952
番茄叶霉病	181	0.907
番茄轮斑病	368	0.927
番茄双斑叶螨	358	0.934
番茄靶斑病	292	0.965
番茄黄化曲叶病毒	1075	0.943
番茄花叶病毒	77	0.903
健康番茄	311	0.906

由测试结果可知，各类别的识别准确率（Precision）均在 0.90 以上，说明模型在目标识别过程中具有较高的识别可信度和较低的误报率，能够有效区分不同病害类型。这一表现达到了系统在非功能性需求中对准确率的设计要求。

5.4 本章小结

本章详细介绍了开发环境配置，并展示了核心功能的具体实现，包括后端 Java 业务逻辑、Flask 模型部署服务以及前端 Vue.js 交互界面的关键代码实现。随后对系统进行了全面的功能性测试和非功能性测试，重点验证了各模块的业务流程正确性、系统响应时延以及模型识别精度等关键指标。测试结果表明，系统成功达到了需求分析阶段设定的功能目标，在响应性能和识别准确率方面均满足预期要求，验证了技术方案的可行性和系统设计的有效性，为实际部署应用提供了可靠保障。

6 结论与展望

6.1 结论

随着农业现代化的发展，农作物生产对智能化、精准化管理的需求日益增长。害虫灾害作为影响农业产量和品质的重要因素，一直是农业生产中的难题。传统的人工识别方式不仅效率低下，且易受主观判断影响，难以满足大规模作业需求。尤其在种植密集区域，病虫害一旦爆发，往往会造成严重损失，因此开发一套高效、准确的智能识别系统势在必行。

本文围绕番茄叶片病虫害问题，分析了害虫图像识别任务的核心需求与挑战，设计并实现了一个集识别、记录管理、用户账号管理等功能于一体的系统平台。系统采用轻量化的 YOLOv8 模型作为识别核心，并结合 SpringBoot、Vue3 及 Flask 构建前后端架构，实现了图像上传、识别结果展示与数据持久化等关键模块，显著提升了识别效率和用户使用体验，为推动农业信息化管理提供了技术支持与实践基础。

本文的重点研究内容如下：

1) 鉴于当前我国农业生产中番茄叶部病害识别自动化程度不足的现状，本研究将番茄叶片病虫害检测确定为核心研究方向。采用基于卷积神经网络的特征学习机制构建目标检测框架，旨在增强模型对微小病斑目标的捕获能力以及对复杂田间环境背景噪声的鲁棒性。研究选用 Ultralytics 公司开发的 YOLOv8 算法作为骨干网络，该算法继承了 YOLO 架构第八代迭代的技术优势，延续了端到端单阶段检测的设计理念，可在一次前向传播过程中并行完成目标定位与分类任务，确保了实时应用场景中的检测效率与精度平衡。结合移动端设备资源限制以及 YOLOv8n 变体在实验数据集上的优异性能表现，本文提出了双重轻量化优化策略：首先引入 Ghost 卷积模块降低网络参数冗余度，随后实施结构化剪枝技术进一步压缩模型规模。最终构建了计算开销与存储需求大幅减少的 Light-YOLOv8 轻量级检测模型。

2) 立足于实际农业应用需求调研，设计了完整的系统总体框架，将平台功能体系划分为病害识别检索、历史记录维护以及用户信息管理三个主要功能模块。运用统一建模语言（UML）工具构建了详细的类结构图和交互时序图，定义了系统各组件之间的协作机制和信息传递路径，系统性地完成了功能架构设计和数据库结构规划。

3) 依托优化后的检测模型实现了完整的应用系统开发。系统技术架构采用浏览器/服务器（B/S）模式下的前后端解耦设计，服务端基于 Spring Boot 微服务框架结合 MyBatis-Plus 持久层组件进行构建，运用 Java 语言实现业务逻辑核心功能，并集成 MySQL 关系型数据库完成数据持久化存储；客户端采用 Vue.js 3.0 响应式框架配合 Element UI 组件库构建交互界面，同时通过 Flask 轻量级 Web 框架实现深度学习模型的在线部署服务。经过全面的功能验证测试，各子系统模块运行状态稳定可靠。

6.2 展望

在本系统的开发过程中，已初步实现了对农作物病虫害图像的识别、记录与管理等功能，形成了前后端分离、支持模型部署与用户交互的完整流程。然而，系统仍存在一定的可优化空间，主要体现在以下几个方面：

1) 后端并发处理能力

目前系统的后端基于 Springboot+Flask 框架，模型推理与图片处理均为同步阻塞操作，在多用户同时上传识别请求的情况下，容易出现性能瓶颈。因此，未来可考虑引入异步处理与缓存机制，提升前端响应速度；

2) 数据存储

当前用户上传的图片及识别记录存储于本地服务器，对服务器的运存空间存在考验。未来可采用新的存储策略如数据库中只存储图片路径及元数据，减轻数据库压力，搭配对象存储服务（OSS）替代本地文件系统，如阿里云 OSS、MinIO 或 Amazon S3，实现图片数据的弹性存储，减轻服务器存储压力。

3) 模型训练数据进一步扩充

尽管当前模型已能识别多类常见病虫害，但训练样本来源仍有限，覆盖场景较为单一。为了进一步提升模型泛化能力和实际应用效果，未来可考虑增加来自不同种植区域、光照条件、拍摄角度的图像样本，定期使用新数据对模型进行再训练与微调，提升实际识别准确率。对收集到的图片数据的筛选，可引入自动化筛选作为前置清洗，提高管理员的筛选效率。管理员进行图片导出时可完善数据导出机制，增加自动标注，使导出的数据更便于投入训练。

致 谢

炎炎夏日，本科生活即将画上句号。回望初来时，干燥的阳光是这座城市于我最直接的印象。南方人不太适应这里的气候，但也正是在这样的环境中，我逐渐习惯了需要自己不断寻找答案的探索。

首先感谢西安交通大学提供的学习与资源支持，使我得以在专业能力与独立思考方面都有了切实的提升。这段经历虽不总是顺利，却是成长的必经之路。

感谢李晨老师在毕业设计过程中给予的指导与支持。李老师严谨细致的治学态度和精简有力的建议帮助我理清了思路，也让我在探索与实践不断成长。

感谢支持我完成毕设的家人朋友们，虽然我在写论文的时候偶尔“人间蒸发”，但你们从没放弃喊我回归现实。感谢所有开源社区的开发者们，开源精神塑造了我对技术的追求。也感谢所有给予我帮助的老师 and 同学们。有你们在身边，我从来不觉得是一个人在面对困难。还要感谢交大艺陶，在社团捏泥、拉坯、等待烧制，构成了我生活中宁静的锚点。

四年过去了，我还是没适应西安的干燥，但这段经历让我收获不少，也很感激一路上遇到的每一个人。

最后要感谢审阅本论文的评审专家和老师，你们的修改建议能使这篇论文结构更清晰、表达更准确。再次感谢大家！

参考文献

- [1] PUJARI J D, YAKKUNDIMATH R, BYADGI A S. Classification of fungal disease symptoms affected on cereals using color texture features[J]. International Journal of Signal Processing, Image Processing & Pattern Recognition, 2013, 6.
- [2] 张云龙, 袁浩, 张晴晴, 等. 基于颜色特征和差直方图的苹果叶部病害识别方法[D]. 江苏农业科学, 2017, 45(14): 171-174.
- [3] 蒲一鸣. 基于机器学习的水稻病害识别和叶龄检测算法研究[D]. 哈尔滨工业大学, 2019.
- [4] ABDULRIDHA J, EHSANI R, ELRAHMAN A, et al. A remote sensing technique for detecting laurel wilt disease in avocado in presence of other biotic and abiotic stresses[J]. Computers and Electronics in Agriculture, 2019, 156: 549-557.
- [5] CHAKRABORTY S, PAUL S, RAHAT-UZ-ZAMAN M. Prediction of apple leaf diseases using multiclass support vector machine[C]//Proceedings of the 2021 2nd International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST). IEEE, 2021: 147-151.
- [6] ZENG G. Fruit and vegetables classification system using image saliency and convolutional neural network[C]//2017 IEEE 3rd Information technology and mechatronics engineering conference (ITOEC). China: IEEE, 2017: 613-617.
- [7] 毛锐, 张宇晨, 王泽玺, 等. 利用改进 Faster-RCNN 识别小麦条锈病和黄矮病[J]. 农业工程学报, 2022, 38(17): 176-185.
- [8] 石晨宇, 周春, 靳鸿, 等. 基于卷积神经网络的农作物病害识别研究[J]. 国外电子测量技术, 2021.
- [9] 李文逵, 韩俊英. 基于一种轻量级卷积神经网络的植物叶片图像识别研究[J]. 软件工程, 2022, 25(02): 10-13+9. DOI:10.19644/j.cnki.issn2096-1472.2022.002.003.
- [10] LIU W. SSD: Single shot multibox detector[C] //Computer Vision & Pattern Recognition. USA: IEEE, 2016: 21-37.
- [11] REDMON J, DIVVALA S, GIRSHICK R, et al. You only look once: Unified, real-time object detection[EB/OL]. (2015-06-02) [2023-12-01]. <https://doi.org/10.48550/arXiv.1506.02640>.
- [12] CHEN Y, JIAO M, PENG X, et al. Study on positioning and detection of crayfish body parts based on machine vision[J]. Journal of Food Measurement and Characterization, 2024, 18(7): 1-13.
- [13] KARTHIK R, VARDHAN G V, KHAITAN S, et al. A dual-track feature fusion model utilizing Group Shuffle Residual DeformNet and Swin Transformer for the classification of grape leaf diseases[J]. Scientific Reports, 2024, 14(1): 14510.
- [14] LI Z, KANG L, RAO H, et al. Camellia oleifera fruit detection algorithm in natural environment based on lightweight convolutional neural network[J]. Applied Sciences, 2023, 13(18): 10394.
- [15] XIAO J, KANG G, WANG L, et al. Real-time lightweight detection of lychee diseases with enhanced YOLOV7 and edge computing[J]. Agronomy, 2023, 13, 2866.
- [16] 谭厚森, 马文宏, 田原, 等. 基于改进 YOLOv8n 的香梨目标检测方法[J]. 农业工程学报, 2024, 40(11): 178-185.
- [17] LI J, LIU Y, LI C, et al. Pineapple detection with yolov7-tiny network model improved via pruning and a lightweight backbone sub-network[J]. Remote Sensing, 2024, 16(15): 2805.
- [18] Shrestha A, Mahmood A. Review of deep learning algorithms and architectures[J]. IEEE Access, 2019,

- 7: 53040-53065.
- [19] Girshick R, Donahue J, Darrell T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2014: 580-587.
 - [20] He K M., Zhang X Y., Ren S Q., et al. Spatial pyramid pooling in deep convolutional networks for visual recognition[J]. IEEE Trans. Pattern Analysis and Machine Intelligence, 2015, 37(9): 1904-1916.
 - [21] Girshick R. Fast r-cnn[C]//Proceedings of the IEEE international conference on computer vision. 2015: 1440-1448.
 - [22] Ren S, He K, Girshick R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks[J]. Advances in Neural Information Processing Systems, 2015, 28.
 - [23] Dai J., Li Y., He K., et al. R-FCN: object detection via region-based fully convolutional networks[A], in: Advances in Neural Information Processing Systems[C], 2016, 379-387.
 - [24] VARGHESE R, SAMBATH M. YOLOv8: A novel object detection algorithm with enhanced performance and robustness[C]//2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS). Chennai: IEEE, 2024: 1-6.
 - [25] 马盼,杨子恒,万虎,等.基于 YOLOv8 网络的棉蚜图像识别算法及软件系统设计[J].智能化农业装备学报(中英文),2023,4(03):42-49.
 - [26] 石浩,陈进,李耀明,朱亚辉,陈宇航,杨平越. 基于改进 YOLOv8-seg 模型的水稻籽粒图像精准分割方法[J/OL]. 南京农业大学学报.<https://link.cnki.net/urlid/32.1148.S.20250522.1411.007>
 - [27] 任梦真, 罗雨竹, 黄英来. 改进 YOLOv8n 的蓝莓冠层果实成熟度识别方法 [J/OL] . 哈尔滨理工大学学报. <https://link.cnki.net/urlid/23.1404.N.20250522.0916.004>
 - [28] Han K, Wang Y, Tian Q, et al. Ghostnet: More features from cheap operations[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2020: 1580-1589.
 - [29] YANG W, MA X, HU W C, et al. Lightweight blueberry fruit recognition based on multi-scale and attention fusion NCBAM[J]. Agronomy, 2022, 12(10): 2354.
 - [30] 景亮, 鲍致远. 基于 Ghost-CA-YOLOv4 的果园障碍物检测算法[J]. 软件导刊, 2023, 22(11): 180-185.
 - [31] LI H, KADAV A, DURDURAN A, et al. Pruning filters for efficient ConvNets[C]// International Conference on Learning Representations (ICLR). 2017.
 - [32] SEBASTIAN PALACIO BETANCUR. PlantVillage for object detection YOLO[DB/OL]. Kaggle, 2024. [2025-05-23]. <https://doi.org/10.34740/KAGGLE/DS/5363572>.
 - [33] 李燕燕. 基于 MVC 框架的题库管理系统的设计与实现 [J] . 中国现代教育装备, 2022, No.395(19): 16-18. DOI:10.13492/j.cnki.cmee.2022.19.026.
 - [34] 陆地, 储蓄蓄, 陶静联, 等. 基于阿里云 OSS 的传统文件中心改造 [J] . 有线电视技术, 2019(6).
 - [35] 叶刚, 王立河, 王英明, 等. 基于 Mybatis Plus 的动态生成代码设计与实现 [J] . 电脑编程技巧与维护, 2019(7): 7-8.